

TalentLens: Recruitment & Onboarding Codebook

This analyses of recruitment and onboarding data is from Pegasus Mortgage Lending Center, The data set is used to explore how Business Analytics can improve the efficiency and effectiveness of the organisation's recruitment process.

The dataset contains 31,500 unique candidate records from the 2020 - 2026 covering information such as candidate characteristics, recruitment sources, progression through recruitment stages, communication activities, licensing information and hiring outcomes.

By applying descriptive analysis, exploratory data analysis, recruitment funnel analysis and predictive modelling, the dissertation aims to identify patterns in candidate progression, understand where candidates are lost during the recruitment process, and assess whether historical data can support better decision-making.

The analysis also considers important data-quality issues, including substantial missingness and inconsistencies between recruitment-status fields, to ensure that the findings are interpreted responsibly. Ultimately, the purpose of the dissertation is to provide evidence-based and practical recommendations that can help Pegasus Mortgage Lending Center improve recruitment processes, while recognising the limitations and risks associated with using historical recruitment data.

```
In [1]: # Import the libraries and the dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score

import scipy.cluster.hierarchy as sch
from sklearn.cluster import AgglomerativeClustering, KMeans

from sklearn.decomposition import PCA
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix, classification_report

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    average_precision_score, f1_score
)

RANDOM_STATE = 42
```

```
In [3]: import os
        print(os.getcwd())
```

C:\Users\REGAL COMPUTER\OneDrive\Desktop\Dissertation\Final Submission

```
In [2]: import warnings
        warnings.filterwarnings('ignore')
```

```
In [4]: #Load the dataset
        df = pd.read_csv("C:\\Users\\REGAL COMPUTER\\OneDrive\\Desktop\\Dissertation\\Full Draft\\Clean Dataset.csv")
```

Data Preparation

```
In [5]: df.head()
```

Out[5]:

	Candidate ID.1	Gender	Email	Phone Number	Mobile Number	City	Country	Created Time	Modified Time	Currency	...	Work Type	Licensi Stat
0	ZR_28480_CAND	Male	Yes	No	Yes	NORTH YORK	Canada	12/29/2023 9:31	3/6/2026 13:01	CAD	...	NaN	Authoriz to S
1	ZR_28476_CAND	Female	Yes	Yes	No	Cambridge	Canada	12/20/2023 16:31	5/6/2026 14:43	CAD	...	Full Time	Expir
2	ZR_28475_CAND	Male	Yes	No	Yes	Brampton	Canada	12/20/2023 16:29	5/6/2026 14:43	CAD	...	NaN	N
3	ZR_28474_CAND	Male	Yes	No	Yes	Toronto	Canada	12/19/2023 8:54	5/6/2026 14:43	CAD	...	Part Time	N
4	ZR_28473_CAND	Male	Yes	No	Yes	Cambridge	Canada	12/19/2023 8:54	5/6/2026 14:43	CAD	...	NaN	Authoriz to S

5 rows × 45 columns



In [6]: `df.shape`

Out[6]: (1048575, 45)

In [7]: `df.info()`

<class 'pandas.DataFrame'>

RangeIndex: 1048575 entries, 0 to 1048574

Data columns (total 45 columns):

#	Column	Non-Null Count	Dtype
0	Candidate ID.1	31500 non-null	str
1	Gender	31500 non-null	str
2	Email	31500 non-null	str
3	Phone Number	31500 non-null	str
4	Mobile Number	31500 non-null	str
5	City	9361 non-null	str
6	Country	31500 non-null	str
7	Created Time	31500 non-null	str
8	Modified Time	31500 non-null	str
9	Currency	31500 non-null	str
10	Last Activity Time	31500 non-null	str
11	Last emailed	11228 non-null	str
12	Source	31500 non-null	str
13	Email Opt Out	31500 non-null	str
14	Is Hot Candidate	31500 non-null	str
15	Is Locked	31500 non-null	str
16	Candidate Status	31500 non-null	str
17	Initial Contact Date	31500 non-null	str
18	Institute 1 Sign Up Date	5458 non-null	str
19	Hire Date	1716 non-null	str
20	Ethnicity	31500 non-null	str
21	Religion	31500 non-null	str
22	Position Category	31500 non-null	str
23	License Class	17628 non-null	str
24	License year	22971 non-null	float64
25	Years of Experience	22972 non-null	float64
26	Current and Previous Mortgage	13694 non-null	str
27	Equifax Credit Score	31498 non-null	str
28	Criminal History	1401 non-null	str
29	Prior Discipline Action by a Regulator	8 non-null	str
30	Professional License Suspension	1508 non-null	str
31	Candidate Stage	27914 non-null	str
32	Realtor License	1452 non-null	str
33	Hiring Validation	31500 non-null	str
34	1 Month Fee Waiver	1420 non-null	str
35	Work Type	1191 non-null	str
36	Licensing Status	19285 non-null	str

37	Province	31137	non-null	str
38	Parked	31419	non-null	str
39	Pre-Interview Info Email	31419	non-null	str
40	Post Interview Info Email	31419	non-null	str
41	Discussion Type	1990	non-null	str
42	Live Transfer	1989	non-null	str
43	Discussion Stage	902265	non-null	str
44	Schedule Discussion	31419	non-null	str

dtypes: float64(2), str(43)
memory usage: 377.7 MB

```
In [8]: #Converting Dates from string to Datetime object
date_columns = [
    'Created Time',
    'Modified Time',
    'Hire Date',
    'Initial Contact Date'
]

for col in date_columns:
    df[col] = pd.to_datetime(df[col], errors='coerce')
```

```
In [9]: # Encoding Yes & No Columns to Binary code
yes_no_columns = ['Email',
                  'Phone Number',
                  'Mobile Number',
                  'Email Opt Out',
                  'Is Locked',
                  'Equifax Credit Score',
                  'Hiring Validation',
                  'Schedule Discussion',
                  'Parked',
                  'Pre-Interview Info Email',
                  'Post Interview Info Email'
                ]

for col in yes_no_columns:
    df[col + '_Encoded'] = df[col].map({
        'Yes': 1,
        'No': 0
    })
```

```
In [10]: df.info()
```

<class 'pandas.DataFrame'>

RangeIndex: 1048575 entries, 0 to 1048574

Data columns (total 56 columns):

#	Column	Non-Null Count	Dtype
0	Candidate ID.1	31500 non-null	str
1	Gender	31500 non-null	str
2	Email	31500 non-null	str
3	Phone Number	31500 non-null	str
4	Mobile Number	31500 non-null	str
5	City	9361 non-null	str
6	Country	31500 non-null	str
7	Created Time	31500 non-null	datetime64[us]
8	Modified Time	31500 non-null	datetime64[us]
9	Currency	31500 non-null	str
10	Last Activity Time	31500 non-null	str
11	Last emailed	11228 non-null	str
12	Source	31500 non-null	str
13	Email Opt Out	31500 non-null	str
14	Is Hot Candidate	31500 non-null	str
15	Is Locked	31500 non-null	str
16	Candidate Status	31500 non-null	str
17	Initial Contact Date	23959 non-null	datetime64[us]
18	Institute 1 Sign Up Date	5458 non-null	str
19	Hire Date	1716 non-null	datetime64[us]
20	Ethnicity	31500 non-null	str
21	Religion	31500 non-null	str
22	Position Category	31500 non-null	str
23	License Class	17628 non-null	str
24	License year	22971 non-null	float64
25	Years of Experience	22972 non-null	float64
26	Current and Previous Mortgage	13694 non-null	str
27	Equifax Credit Score	31498 non-null	str
28	Criminal History	1401 non-null	str
29	Prior Discipline Action by a Regulator	8 non-null	str
30	Professional License Suspension	1508 non-null	str
31	Candidate Stage	27914 non-null	str
32	Realtor License	1452 non-null	str
33	Hiring Validation	31500 non-null	str
34	1 Month Fee Waiver	1420 non-null	str
35	Work Type	1191 non-null	str
36	Licensing Status	19285 non-null	str

37	Province	31137	non-null	str
38	Parked	31419	non-null	str
39	Pre-Interview Info Email	31419	non-null	str
40	Post Interview Info Email	31419	non-null	str
41	Discussion Type	1990	non-null	str
42	Live Transfer	1989	non-null	str
43	Discussion Stage	902265	non-null	str
44	Schedule Discussion	31419	non-null	str
45	Email_Encoded	31500	non-null	float64
46	Phone Number_Encoded	31500	non-null	float64
47	Mobile Number_Encoded	31500	non-null	float64
48	Email Opt Out_Encoded	31500	non-null	float64
49	Is Locked_Encoded	31500	non-null	float64
50	Equifax Credit Score_Encoded	31498	non-null	float64
51	Hiring Validation_Encoded	31500	non-null	float64
52	Schedule Discussion_Encoded	31419	non-null	float64
53	Parked_Encoded	31419	non-null	float64
54	Pre-Interview Info Email_Encoded	31419	non-null	float64
55	Post Interview Info Email_Encoded	31419	non-null	float64

dtypes: datetime64[us](4), float64(13), str(39)

memory usage: 464.0 MB

```
In [11]: #Checking Missing Values
df.isnull().sum()
```


Out[11]:	Candidate ID.1	1017075
	Gender	1017075
	Email	1017075
	Phone Number	1017075
	Mobile Number	1017075
	City	1039214
	Country	1017075
	Created Time	1017075
	Modified Time	1017075
	Currency	1017075
	Last Activity Time	1017075
	Last emailed	1037347
	Source	1017075
	Email Opt Out	1017075
	Is Hot Candidate	1017075
	Is Locked	1017075
	Candidate Status	1017075
	Initial Contact Date	1024616
	Institute 1 Sign Up Date	1043117
	Hire Date	1046859
	Ethnicity	1017075
	Religion	1017075
	Position Category	1017075
	License Class	1030947
	License year	1025604
	Years of Experience	1025603
	Current and Previous Mortgage	1034881
	Equifax Credit Score	1017077
	Criminal History	1047174
	Prior Discipline Action by a Regulator	1048567
	Professional License Suspension	1047067
	Candidate Stage	1020661
	Realtor License	1047123
	Hiring Validation	1017075
	1 Month Fee Waiver	1047155
	Work Type	1047384
	Licensing Status	1029290
	Province	1017438
	Parked	1017156
	Pre-Interview Info Email	1017156
	Post Interview Info Email	1017156
	Discussion Type	1046585

Live Transfer	1046586
Discussion Stage	146310
Schedule Discussion	1017156
Email_Encoded	1017075
Phone Number_Encoded	1017075
Mobile Number_Encoded	1017075
Email Opt Out_Encoded	1017075
Is Locked_Encoded	1017075
Equifax Credit Score_Encoded	1017077
Hiring Validation_Encoded	1017075
Schedule Discussion_Encoded	1017156
Parked_Encoded	1017156
Pre-Interview Info Email_Encoded	1017156
Post Interview Info Email_Encoded	1017156
dtype: int64	

```
In [12]: df = df[df["Candidate ID.1"].notna()].copy() # keep rows with candidate ID only
```

```
In [13]: df.isnull().sum()
```

Out[13]:	Candidate ID.1	0
	Gender	0
	Email	0
	Phone Number	0
	Mobile Number	0
	City	22139
	Country	0
	Created Time	0
	Modified Time	0
	Currency	0
	Last Activity Time	0
	Last emailed	20272
	Source	0
	Email Opt Out	0
	Is Hot Candidate	0
	Is Locked	0
	Candidate Status	0
	Initial Contact Date	7541
	Institute 1 Sign Up Date	26042
	Hire Date	29784
	Ethnicity	0
	Religion	0
	Position Category	0
	License Class	13872
	License year	8529
	Years of Experience	8528
	Current and Previous Mortgage	17806
	Equifax Credit Score	2
	Criminal History	30099
	Prior Discipline Action by a Regulator	31492
	Professional License Suspension	29992
	Candidate Stage	3586
	Realtor License	30048
	Hiring Validation	0
	1 Month Fee Waiver	30080
	Work Type	30309
	Licensing Status	12215
	Province	363
	Parked	81
	Pre-Interview Info Email	81
	Post Interview Info Email	81
	Discussion Type	29510

Live Transfer	29511
Discussion Stage	29512
Schedule Discussion	81
Email_Encoded	0
Phone Number_Encoded	0
Mobile Number_Encoded	0
Email Opt Out_Encoded	0
Is Locked_Encoded	0
Equifax Credit Score_Encoded	2
Hiring Validation_Encoded	0
Schedule Discussion_Encoded	81
Parked_Encoded	81
Pre-Interview Info Email_Encoded	81
Post Interview Info Email_Encoded	81

dtype: int64

```
In [14]: categorical_columns = df.select_dtypes(include='object').columns
```

```
for col in categorical_columns:
    df[col] = df[col].fillna('Unknown')
```

```
In [15]: numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns
```

```
for col in numerical_columns:
    df[col] = df[col].fillna(df[col].median())
```

```
In [16]: date_columns = ['Created Time', 'Modified Time']
```

```
for col in date_columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

```
In [17]: df.replace(r'^\s*$', np.nan, regex=True, inplace=True)
```

Out[17]:

	Candidate ID.1	Gender	Email	Phone Number	Mobile Number	City	Country	Created Time	Modified Time	Currency	...	Number_End
0	ZR_28480_CAND	Male	Yes	No	Yes	NORTH YORK	Canada	2023-12-29 09:31:00	2026-03-06 13:01:00	CAD	...	
1	ZR_28476_CAND	Female	Yes	Yes	No	Cambridge	Canada	2023-12-20 16:31:00	2026-05-06 14:43:00	CAD	...	
2	ZR_28475_CAND	Male	Yes	No	Yes	Brampton	Canada	2023-12-20 16:29:00	2026-05-06 14:43:00	CAD	...	
3	ZR_28474_CAND	Male	Yes	No	Yes	Toronto	Canada	2023-12-19 08:54:00	2026-05-06 14:43:00	CAD	...	
4	ZR_28473_CAND	Male	Yes	No	Yes	Cambridge	Canada	2023-12-19 08:54:00	2026-05-06 14:43:00	CAD	...	
...	...	...	...	...	...	...	...	...	...	...	...	
31495	ZR_28486_CAND	Male	Yes	No	Yes	Brampton	Canada	2024-01-02 12:13:00	2025-03-10 16:30:00	CAD	...	
31496	ZR_28485_CAND	Female	Yes	Yes	No	Brampton, ON	Canada	2024-01-02 12:13:00	2026-02-19 11:11:00	CAD	...	
31497	ZR_28484_CAND	Male	Yes	No	Yes	Richmond Hill	Canada	2024-01-02 12:13:00	2025-09-10 16:38:00	CAD	...	
31498	ZR_28483_CAND	Female	Yes	No	Yes	Scarborough	Canada	2024-01-02 12:13:00	2026-02-18 14:46:00	CAD	...	
31499	ZR_28482_CAND	Female	Yes	No	Yes	AYR	Canada	2024-01-02	2025-03-10	CAD	...	

Candidate ID.1	Gender	Email	Phone Number	Mobile Number	City	Country	Created Time	Modified Time	Currency	...	Number_End	I
							12:13:00	16:30:00				

31500 rows × 56 columns

```
In [18]: categorical_columns = df.select_dtypes(include='object').columns

for col in categorical_columns:
    df[col] = df[col].fillna('Unknown')
```

```
In [19]: df.isnull().sum()
```

Out[19]:	Candidate ID.1	0
	Gender	0
	Email	0
	Phone Number	0
	Mobile Number	0
	City	0
	Country	0
	Created Time	0
	Modified Time	0
	Currency	0
	Last Activity Time	0
	Last emailed	0
	Source	0
	Email Opt Out	0
	Is Hot Candidate	0
	Is Locked	0
	Candidate Status	0
	Initial Contact Date	7541
	Institute 1 Sign Up Date	0
	Hire Date	29784
	Ethnicity	0
	Religion	0
	Position Category	0
	License Class	0
	License year	0
	Years of Experience	0
	Current and Previous Mortgage	0
	Equifax Credit Score	0
	Criminal History	0
	Prior Discipline Action by a Regulator	0
	Professional License Suspension	0
	Candidate Stage	0
	Realtor License	0
	Hiring Validation	0
	1 Month Fee Waiver	0
	Work Type	0
	Licensing Status	0
	Province	0
	Parked	0
	Pre-Interview Info Email	0
	Post Interview Info Email	0
	Discussion Type	0

Live Transfer	0
Discussion Stage	0
Schedule Discussion	0
Email_Encoded	0
Phone Number_Encoded	0
Mobile Number_Encoded	0
Email Opt Out_Encoded	0
Is Locked_Encoded	0
Equifax Credit Score_Encoded	0
Hiring Validation_Encoded	0
Schedule Discussion_Encoded	0
Parked_Encoded	0
Pre-Interview Info Email_Encoded	0
Post Interview Info Email_Encoded	0
dtype: int64	

In [20]: `df.shape`

Out[20]: (31500, 56)

In [21]: `df.duplicated().sum()`

Out[21]: np.int64(0)

In [22]: *#To Trim and Standardize the Dataset*
`df.select_dtypes(include='object').columns`

Out[22]: Index(['Candidate ID.1', 'Gender', 'Email', 'Phone Number', 'Mobile Number',
'City', 'Country', 'Currency', 'Last Activity Time', 'Last emailed',
'Source', 'Email Opt Out', 'Is Hot Candidate', 'Is Locked',
'Candidate Status', 'Institute 1 Sign Up Date', 'Ethnicity', 'Religion',
'Position Category', 'License Class', 'Current and Previous Mortgage',
'Equifax Credit Score', 'Criminal History',
'Prior Discipline Action by a Regulator',
'Professional License Suspension', 'Candidate Stage', 'Realtor License',
'Hiring Validation', '1 Month Fee Waiver', 'Work Type',
'Licensing Status', 'Province', 'Parked', 'Pre-Interview Info Email',
'Post Interview Info Email', 'Discussion Type', 'Live Transfer',
'Discussion Stage', 'Schedule Discussion'],
dtype='str')


```
In [23]: for col in df.select_dtypes(include='object').columns:  
         print(f"\n{col}")  
         print(df[col].unique())
```

Candidate ID.1

<ArrowStringArray>

```
['ZR_28480_CAND', 'ZR_28476_CAND', 'ZR_28475_CAND', 'ZR_28474_CAND',  
 'ZR_28473_CAND', 'ZR_28472_CAND', 'ZR_28471_CAND', 'ZR_28470_CAND',  
 'ZR_28469_CAND', 'ZR_28468_CAND',  
 ...  
 'ZR_28492_CAND', 'ZR_28491_CAND', 'ZR_28490_CAND', 'ZR_28489_CAND',  
 'ZR_28487_CAND', 'ZR_28486_CAND', 'ZR_28485_CAND', 'ZR_28484_CAND',  
 'ZR_28483_CAND', 'ZR_28482_CAND']
```

Length: 31500, dtype: str

Gender

<ArrowStringArray>

```
['Male', 'Female']
```

Length: 2, dtype: str

Email

<ArrowStringArray>

```
['Yes', 'No']
```

Length: 2, dtype: str

Phone Number

<ArrowStringArray>

```
['No', 'Yes']
```

Length: 2, dtype: str

Mobile Number

<ArrowStringArray>

```
['Yes', 'No']
```

Length: 2, dtype: str

City

<ArrowStringArray>

```
['NORTH YORK',  
  

```

```
'Cambridge',  
  

```

```
'Brampton',  
  

```

```
'Toronto',  
  

```

```
'Oshawa',  
  
'Hamilton',  
  
'Ancaster',  
  
'Innisfil',  
  
'AJAX',  
  
'Unknown',  
...  
  
'newmarket',  
  
'Hamilton, ON',  
  
'Drumbo',  
  
'Clarence Creek',  
  
'Delta',  
  
'Cambridge ON',  
  
'Cookstown',  
  
'Guleph',  
'Hamilton (West Kentley / McQuesten / Parkview / Hamilton Beach / East Industrial Sector / Normanhurst / Homeside /  
East Crown Point)',  
  
'Brampton ,ON']  
Length: 905, dtype: str
```

```
Country  
<ArrowStringArray>  
['Canada']  
Length: 1, dtype: str
```

```
Currency  
<ArrowStringArray>  
['CAD', 'PKR', 'INR']
```

Length: 3, dtype: str

Last Activity Time

<ArrowStringArray>

```
[ '3/6/2026 13:01', '5/6/2026 14:43', '2/10/2026 15:40',  
  '4/15/2026 17:30', '3/13/2026 12:00', '3/30/2026 11:36',  
  '11/14/2025 12:59', '1/9/2026 9:59', '9/23/2025 12:54',  
  '5/9/2025 11:55',  
  ...  
  '3/16/2026 16:57', '2/4/2026 12:33', '4/27/2026 17:12',  
  '3/25/2026 16:15', '10/20/2025 12:33', '3/18/2026 10:50',  
  '4/13/2026 17:51', '4/29/2026 11:03', '3/21/2025 16:16',  
  '3/31/2026 15:41']
```

Length: 14003, dtype: str

Last emailed

<ArrowStringArray>

```
[ 'Unknown', '1/5/2024 13:35', '1/9/2024 13:02',  
  '11/5/2024 15:19', '12/21/2023 14:30', '12/20/2023 16:09',  
  '12/20/2023 16:02', '5/2/2024 15:33', '12/20/2023 13:09',  
  '7/18/2024 14:23',  
  ...  
  '1/10/2024 11:13', '7/8/2025 15:37', '9/25/2024 16:22',  
  '1/16/2024 15:33', '1/17/2024 14:35', '5/8/2024 15:39',  
  '1/22/2024 12:52', '1/14/2025 12:24', '1/2/2024 14:00',  
  '1/18/2024 9:06']
```

Length: 10242, dtype: str

Source

<ArrowStringArray>

```
[ 'Referral', 'Institute 1', 'Call-In', 'Emailed',  
  'Website', 'Social Media 1', 'BI', 'MCD',  
  'Institute 3', 'Cold Call', 'Institute 2', 'Existing Agent',  
  'Search Engine 2', 'Resume Inbox', 'Market Place', 'Job Site',  
  'Social Media 2', 'CareerSite', 'Walk-In', 'Trade Show',  
  'Emailed-In', 'Job Site Resume', 'Social Media', 'Mass Email',  
  'Institute 4', 'Unkown']
```

Length: 26, dtype: str

Email Opt Out

<ArrowStringArray>

```
['No', 'Yes']
```

Length: 2, dtype: str

Is Hot Candidate

<ArrowStringArray>

['No', 'Yes']

Length: 2, dtype: str

Is Locked

<ArrowStringArray>

['Yes', 'No']

Length: 2, dtype: str

Candidate Status

<ArrowStringArray>

['Hired', 'Joined Another Brokerage',
 'No-Show', 'Interview-Scheduled',
 'Rejected by hiring manager', 'Attempted to Contact',
 'Following Up to Join', 'On-Hold',
 'Junk candidate', 'Non Responsive',
 'Candidate Refused to Meet', 'Candidate Refused to Join',
 'Associated', 'Interview-to-be-Scheduled',
 'Unqualified', 'Pre-Course',
 'New', 'Joined',
 'Offer-Withdrawn', 'Offer-Made']

Length: 20, dtype: str

Institute 1 Sign Up Date

<ArrowStringArray>

['Unknown', '12/19/2023', '12/15/2023', '12/16/2023', '12/18/2023',
 '12/14/2023', '12/12/2023', '12/13/2023', '12/11/2023', '12/8/2023',
 ...
 '12/29/2023', '12/30/2023', '12/31/2023', '1/1/2024', '12/20/2023',
 '12/21/2023', '12/22/2023', '12/23/2023', '12/24/2023', '12/25/2023']

Length: 1677, dtype: str

Ethnicity

<ArrowStringArray>

['South Asian', 'Caucasian', 'African',
 'Middle Eastern', 'Black', 'South-East Asian',
 'Latin American', 'Hispanic', 'East Asian',
 'Caribbean', 'Indigenous', 'Unknown',
 'European', 'Pacific Islander']

Length: 14, dtype: str

Religion

<ArrowStringArray>

```
[      'Hinduism',      'Christianity',  
      'Sikhism',      'Islam',  
      'Buddhism',      'Unknown',  
      'Judaism',      'Atheist',  
      'Unkno', 'Christianity (Orthodox)',  
      'Jainism',      'Zoroastrianism']
```

Length: 12, dtype: str

Position Category

<ArrowStringArray>

```
[      'Mortgage Agent',      'Principal Broker',      'Broker',  
      'Agent', 'Mortgage Professional', 'Referral Partner (RPP)',  
      'Realtor']
```

Length: 7, dtype: str

License Class

<ArrowStringArray>

```
[      'Unknown',      'Agent Level 1',      'Agent Level 2',  
      'Agent',      'Broker', 'Principal Broker']
```

Length: 6, dtype: str

Current and Previous Mortgage

<ArrowStringArray>

```
[  
'X',  
  
'Unknown',  
  
'Affinity Mortgage Solutions Inc.',  
  
'Mountainview Mortgage Inc.',  
  
'AKAL Mortgages Inc.',  
  
'MORTGAGE ALLIANCE',  
  
'BRX Mortgage Inc.',
```

'Unknown',

'Highway Traffic Act Violation',
 'Criminal Background',
 'Driving Under Influence',
 'Criminal Background;Not Applicable',
 'Driving Under Influence;Criminal Background']
Length: 7, dtype: str

Prior Discipline Action by a Regulator
<ArrowStringArray>
['Unknown', 'Yes']
Length: 2, dtype: str

Professional License Suspension
<ArrowStringArray>
['Not Applicable', 'Unknown']
Length: 2, dtype: str

Candidate Stage
<ArrowStringArray>
['In Review', 'New', 'Unknown', 'Hired', 'Engaged', 'Available']
Length: 6, dtype: str

Realtor License
<ArrowStringArray>
['No', 'Unknown', 'Yes']
Length: 3, dtype: str

Hiring Validation
<ArrowStringArray>
['Yes', 'No']
Length: 2, dtype: str

1 Month Fee Waiver
<ArrowStringArray>
['No', 'Unknown', 'Yes', 'Not Applicable']
Length: 4, dtype: str

Work Type
<ArrowStringArray>
['Unknown', 'Full Time', 'Part Time']
Length: 3, dtype: str

Licensing Status

<ArrowStringArray>

```
[ 'Authorized to Sell',      'Expired',      'Unknown',  
  'Not Authorized to Sell',  'Surrendered', 'Pending',  
                                'Revoked',         'Suspended',    'Application Refused',  
                                'Non-Active']
```

Length: 10, dtype: str

Province

<ArrowStringArray>

```
[ 'Ontario',      'Unknown',  
  'Alberta',      'ON',  
  'New Brunswick', 'British Columbia',  
  'Saskatchewan', 'Manitoba',  
  'Nova Scotia',   'Prince Edward Island',  
  'Newfoundland and Labrador', 'Quebec',  
  'Alaska',        'Florida',  
  'Michigan']
```

Length: 15, dtype: str

Parked

<ArrowStringArray>

```
['No', 'Unknown', 'Yes']
```

Length: 3, dtype: str

Pre-Interview Info Email

<ArrowStringArray>

```
['No', 'Yes', 'Unknown']
```

Length: 3, dtype: str

Post Interview Info Email

<ArrowStringArray>

```
['No', 'Yes', 'Unknown']
```

Length: 3, dtype: str

Discussion Type

<ArrowStringArray>

```
['Unknown', 'Scheduled', 'Direct Transfer']
```

Length: 3, dtype: str

Live Transfer

<ArrowStringArray>

```
['Unknown', 'Offered', 'Not Offered']  
Length: 3, dtype: str
```

```
Discussion Stage  
<ArrowStringArray>  
['Unknown', 'Stage 1', 'Stage 2']  
Length: 3, dtype: str
```

```
Schedule Discussion  
<ArrowStringArray>  
['No', 'Yes', 'Unknown']  
Length: 3, dtype: str
```

```
In [24]: categorical_columns = df.select_dtypes(include='object').columns  
  
for col in categorical_columns:  
    df[col] = df[col].astype(str).str.strip().str.title()
```

```
In [25]: #Checking the Result  
for col in categorical_columns:  
    print(f"\n{col}")  
    print(df[col].value_counts())
```

Candidate ID.1
Candidate ID.1
Zr_28480_Cand 1
Zr_28476_Cand 1
Zr_28475_Cand 1
Zr_28474_Cand 1
Zr_28473_Cand 1
..
Zr_28486_Cand 1
Zr_28485_Cand 1
Zr_28484_Cand 1
Zr_28483_Cand 1
Zr_28482_Cand 1
Name: count, Length: 31500, dtype: int64

Gender
Gender
Male 21994
Female 9506
Name: count, dtype: int64

Email
Email
Yes 25219
No 6281
Name: count, dtype: int64

Phone Number
Phone Number
No 27216
Yes 4284
Name: count, dtype: int64

Mobile Number
Mobile Number
Yes 27843
No 3657
Name: count, dtype: int64

City
City
Unknown

```

22139
Brampton
948
Toronto
873
Calgary
708
Mississauga
629

...
Clarence Creek
1
Cambridge On
1
Guleph
1
Hamilton (West Kentley / Mcquesten / Parkview / Hamilton Beach / East Industrial Sector / Normanhurst / Homeside / Ea
st Crown Point)      1
Brampton ,On
1
Name: count, Length: 680, dtype: int64

Country
Country
Canada      31500
Name: count, dtype: int64

Currency
Currency
Cad      31470
Pkr      19
Inr      11
Name: count, dtype: int64

Last Activity Time
Last Activity Time
4/30/2026 13:50      1801
3/7/2025 8:59        1445
5/6/2026 14:40        200
5/6/2026 14:43        199
5/6/2026 14:55        199

```

```

...
10/20/2025 12:33      1
3/18/2026 10:50      1
4/29/2026 11:03      1
3/21/2025 16:16      1
3/31/2026 15:41      1
Name: count, Length: 14003, dtype: int64

```

```

Last emailed
Last emailed
Unknown              20272
2/21/2025 14:23      9
10/31/2023 9:49       9
11/21/2023 11:40      7
11/3/2023 9:14        7
...
5/8/2024 15:39        1
1/22/2024 12:52        1
1/14/2025 12:24        1
1/2/2024 14:00         1
1/18/2024 9:06         1
Name: count, Length: 10242, dtype: int64

```

```

Source
Source
Institute 1          9883
Website              9839
Bi                  7299
Institute 3          1311
Mcd                  601
Institute 2           559
Cold Call            554
Institute 4           469
Referral             330
Call-In              173
Emailed              106
Unkown                80
Job Site Resume       70
Job Site              55
Market Place          44
Careersite            44
Social Media 2        41

```

Existing Agent	11
Social Media 1	8
Walk-In	8
Search Engine 2	6
Emailed-In	4
Trade Show	2
Resume Inbox	1
Social Media	1
Mass Email	1

Name: count, dtype: int64

Email Opt Out

Email Opt Out

No	26450
Yes	5050

Name: count, dtype: int64

Is Hot Candidate

Is Hot Candidate

No	31493
Yes	7

Name: count, dtype: int64

Is Locked

Is Locked

No	29985
Yes	1515

Name: count, dtype: int64

Candidate Status

Candidate Status

Associated	9933
New	7596
Attempted To Contact	4025
Hired	1796
Interview-Scheduled	1649
Candidate Refused To Join	1275
Following Up To Join	983
On-Hold	838
No-Show	788
Joined Another Brokerage	709
Junk Candidate	670

Non Responsive	607
Candidate Refused To Meet	455
Interview-To-Be-Scheduled	53
Rejected By Hiring Manager	50
Joined	38
Offer-Withdrawn	24
Pre-Course	6
Offer-Made	3
Unqualified	2

Name: count, dtype: int64

Institute 1 Sign Up Date

Institute 1 Sign Up Date	
Unknown	26042
2/7/2020	21
12/17/2019	21
2/19/2020	20
1/15/2020	18
...	
12/31/2023	1
1/1/2024	1
12/22/2023	1
12/24/2023	1
12/25/2023	1

Name: count, Length: 1677, dtype: int64

Ethnicity

Ethnicity	
Caucasian	11752
South Asian	10901
Middle Eastern	2755
East Asian	2465
Hispanic	926
Black	800
African	676
South-East Asian	539
Unknown	236
Latin American	211
Caribbean	154
European	70
Pacific Islander	10
Indigenous	5

Name: count, dtype: int64

Religion

Religion

Christianity	15343
--------------	-------

Hinduism	5695
----------	------

Islam	3945
-------	------

Sikhism	3102
---------	------

Buddhism	1579
----------	------

Unknown	1188
---------	------

Judaism	393
---------	-----

Atheist	248
---------	-----

Jainism	3
---------	---

Zoroastrianism	2
----------------	---

Unkno	1
-------	---

Christianity (Orthodox)	1
-------------------------	---

Name: count, dtype: int64

Position Category

Position Category

Mortgage Agent	31366
----------------	-------

Realtor	61
---------	----

Agent	46
-------	----

Broker	13
--------	----

Principal Broker	7
------------------	---

Referral Partner (Rpp)	6
------------------------	---

Mortgage Professional	1
-----------------------	---

Name: count, dtype: int64

License Class

License Class

Unknown	13872
---------	-------

Agent Level 1	9432
---------------	------

Agent Level 2	4523
---------------	------

Agent	1462
-------	------

Broker	1451
--------	------

Principal Broker	760
------------------	-----

Name: count, dtype: int64

Current and Previous Mortgage

Current and Previous Mortgage

Unknown

17807

X	782
Mortgage Alliance Company Of Canada Inc.	640
Ma Mortgage Architects Inc.	335
Real Mortgage Associates Inc.	285

...

Forest City Funding	1
Mortgage Bridge Canada	1
Landeal Capital Inc	1
Promise Mortgage And Financial	1
Mortgages On Point	1

Name: count, Length: 1917, dtype: int64

Equifax Credit Score

Equifax Credit Score

No	29568
----	-------

Yes	1930
-----	------

Unknown	2
---------	---

Name: count, dtype: int64

Criminal History

Criminal History

Unknown	30099
---------	-------

Not Applicable	1373
----------------	------

Highway Traffic Act Violation	12
-------------------------------	----

Criminal Background	9
---------------------	---

Driving Under Influence	5
-------------------------	---

Criminal Background;Not Applicable	1
------------------------------------	---

Driving Under Influence;Criminal Background	1
---	---

Name: count, dtype: int64

Prior Discipline Action by a Regulator

Prior Discipline Action by a Regulator

Unknown	31492
---------	-------

Yes	8
-----	---

Name: count, dtype: int64

Professional License Suspension

Professional License Suspension

Unknown	29992
---------	-------

Not Applicable	1508
----------------	------

Name: count, dtype: int64

Candidate Stage
Candidate Stage
New 18356
Engaged 4912
In Review 4537
Unknown 3586
Hired 85
Available 24
Name: count, dtype: int64

Realtor License
Realtor License
Unknown 30567
No 820
Yes 113
Name: count, dtype: int64

Hiring Validation
Hiring Validation
No 30042
Yes 1458
Name: count, dtype: int64

1 Month Fee Waiver
1 Month Fee Waiver
Unknown 30080
No 578
Yes 528
Not Applicable 314
Name: count, dtype: int64

Work Type
Work Type
Unknown 30309
Part Time 631
Full Time 560
Name: count, dtype: int64

Licensing Status
Licensing Status
Authorized To Sell 13392
Unknown 12215

Expired	4406
Not Authorized To Sell	1121
Surrendered	248
Pending	65
Suspended	38
Revoked	13
Application Refused	1
Non-Active	1

Name: count, dtype: int64

Province

Province	
Ontario	27355
On	1653
Alberta	1524
Quebec	368
Unknown	363
British Columbia	189
New Brunswick	18
Saskatchewan	10
Manitoba	7
Nova Scotia	5
Newfoundland And Labrador	3
Prince Edward Island	2
Alaska	1
Florida	1
Michigan	1

Name: count, dtype: int64

Parked

Parked	
No	30011
Yes	1408
Unknown	81

Name: count, dtype: int64

Pre-Interview Info Email

Pre-Interview Info Email	
No	26310
Yes	5109
Unknown	81

Name: count, dtype: int64

```
Post Interview Info Email
Post Interview Info Email
No          30766
Yes         653
Unknown     81
Name: count, dtype: int64
```

```
Discussion Type
Discussion Type
Unknown     29510
Direct Transfer  1597
Scheduled    393
Name: count, dtype: int64
```

```
Live Transfer
Live Transfer
Unknown     29511
Offered     1984
Not Offered    5
Name: count, dtype: int64
```

```
Discussion Stage
Discussion Stage
Unknown     29512
Stage 1     1982
Stage 2      6
Name: count, dtype: int64
```

```
Schedule Discussion
Schedule Discussion
No          29437
Yes         1982
Unknown     81
Name: count, dtype: int64
```

Following the data preparation process, the final analytical dataset consisted of 31,500 unique candidate records and 56 variables, with no duplicate records identified. The preparation involved removing rows without a valid candidate identifier, standardising inconsistent formatting, converting relevant date fields into a consistent datetime format, and encoding Yes/No variables for subsequent analysis. Missing values were also assessed and treated according to the nature of each variable, with categorical variables assigned an "Unknown" category and numerical variables handled using median imputation where appropriate. This

process ensured that the dataset was structured and sufficiently consistent for the subsequent descriptive, exploratory, recruitment funnel, and predictive analyses.

Descriptive Analysis

```
In [26]: # Total Count  
len(df)
```

```
Out[26]: 31500
```

```
In [27]: #Gender Count  
df['Gender'].value_counts()
```

```
Out[27]: Gender  
Male      21994  
Female     9506  
Name: count, dtype: int64
```

```
In [28]: #Bar Chart  
sns.set_theme(style="whitegrid")  
  
plt.figure(figsize=(7,5))  
  
ax = sns.countplot(  
    data=df,  
    x='Gender',  
    hue='Gender',  
    palette='Set2',  
    legend=False,  
    edgecolor='black',  
    linewidth=1.2)  
  
# Title and axis labels  
plt.title('Distribution of Candidates by Gender',  
          fontsize=16,  
          fontweight='bold',  
          pad=15)  
  
plt.xlabel('Gender', fontsize=13, fontweight='bold')
```

```
plt.ylabel('Number of Candidates', fontsize=13, fontweight='bold')

# Add value Labels
for container in ax.containers:
    ax.bar_label(container,
                  fontsize=11,
                  fontweight='bold',
                  padding=3)

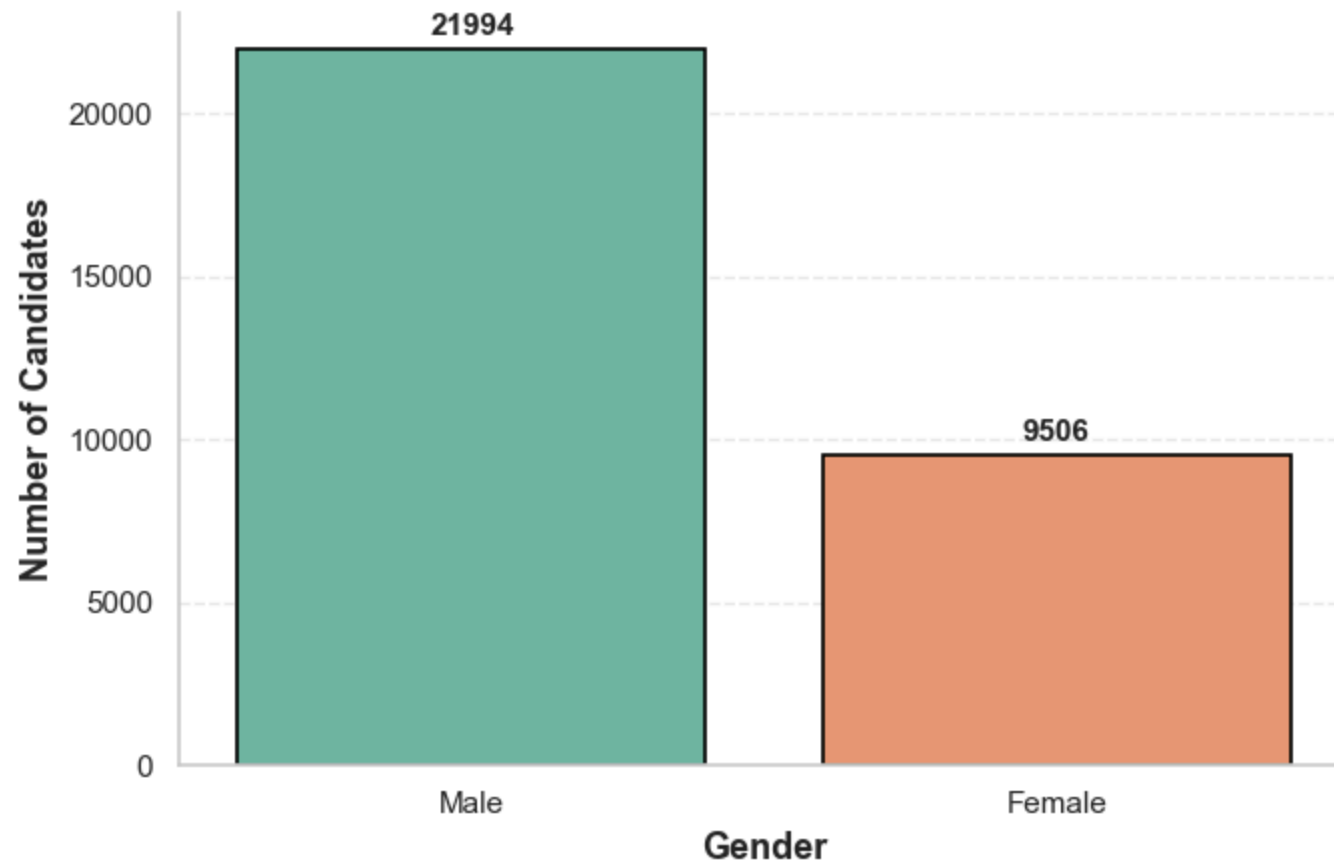
# Improve appearance
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()
```

Distribution of Candidates by Gender



```
In [29]: #Candidate Status  
df['Candidate Status'].value_counts()
```

```
Out[29]: Candidate Status
Associated          9933
New                7596
Attempted To Contact 4025
Hired              1796
Interview-Scheduled 1649
Candidate Refused To Join 1275
Following Up To Join  983
On-Hold            838
No-Show            788
Joined Another Brokerage 709
Junk Candidate      670
Non Responsive      607
Candidate Refused To Meet 455
Interview-To-Be-Scheduled 53
Rejected By Hiring Manager 50
Joined              38
Offer-Withdrawn     24
Pre-Course          6
Offer-Made           3
Unqualified          2
Name: count, dtype: int64
```

```
In [30]: # Bar Chart for Candidate Status
candidate_status = df['Candidate Status'].value_counts()

# Professional style
sns.set_theme(style="whitegrid")

plt.figure(figsize=(11,6))

ax = sns.barplot(
    x=candidate_status.index,
    y=candidate_status.values,
    color='hotpink',
    edgecolor='black',
    linewidth=1.2
)

# Titles
plt.title('Distribution of Candidate Status',
          fontsize=16,
```



```
        fontweight='bold',
        pad=15)

plt.xlabel('Candidate Status',
           fontsize=13,
           fontweight='bold')

plt.ylabel('Number of Candidates',
           fontsize=13,
           fontweight='bold')

# Rotate Labels
plt.xticks(rotation=45,
           ha='right',
           fontsize=11)

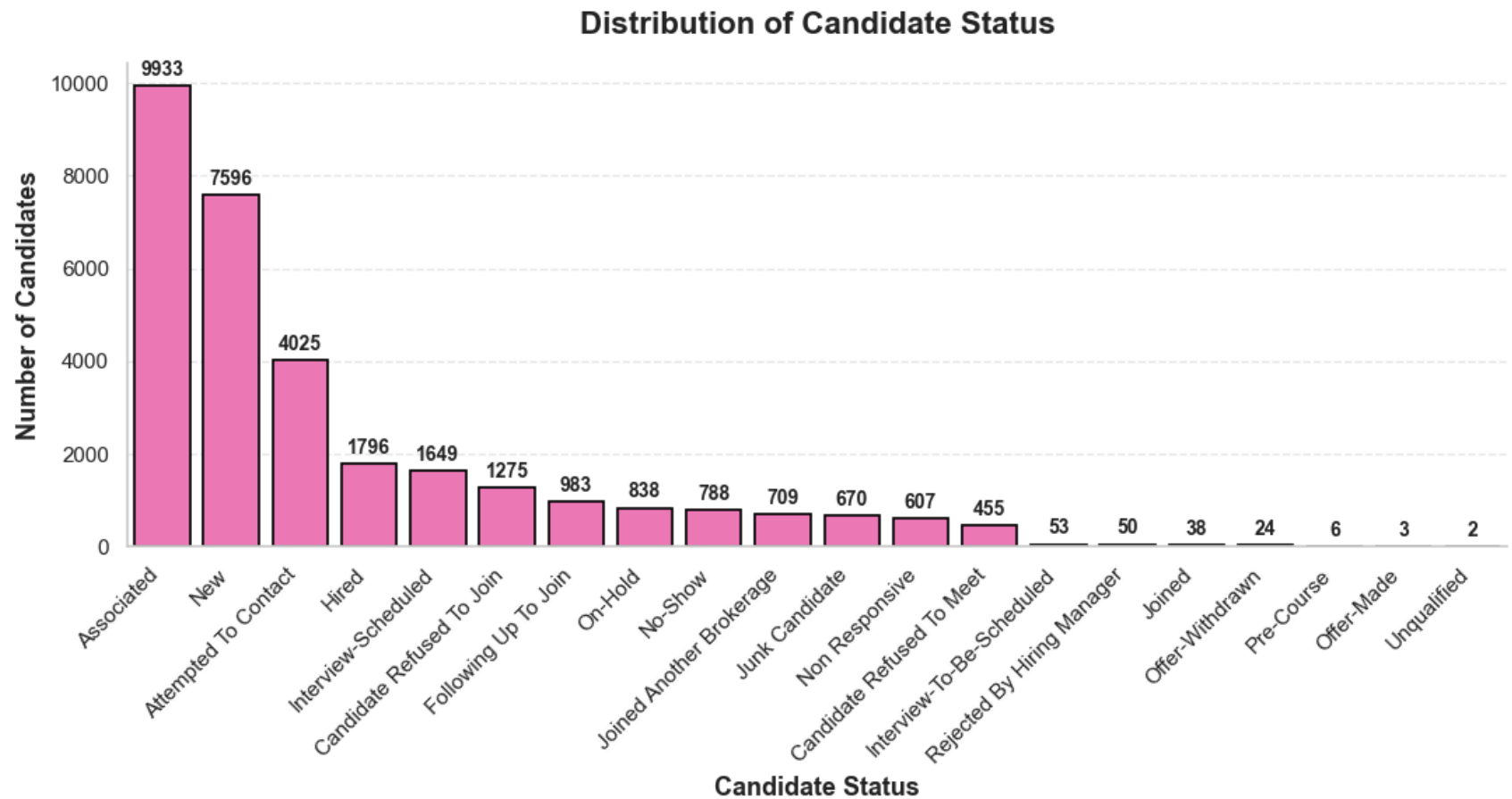
plt.yticks(fontsize=11)

# Add values on bars
for container in ax.containers:
    ax.bar_label(container,
                 fontsize=10,
                 fontweight='bold',
                 padding=3)

# Improve appearance
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()
```



```
In [31]: #Candidate Stage
df['Candidate Stage'].value_counts()
```

```
Out[31]: Candidate Stage
New          18356
Engaged      4912
In Review    4537
Unknown      3586
Hired         85
Available     24
Name: count, dtype: int64
```

```
In [32]: # Bar Chart for Candidate Stage
candidate_stage = df['Candidate Stage'].value_counts()
```

```

# Professional style
sns.set_theme(style="whitegrid")

plt.figure(figsize=(11,6))

ax = sns.barplot(
    x=candidate_stage.index,
    y=candidate_stage.values,
    color='mediumseagreen',
    edgecolor='black',
    linewidth=1.2
)

# Title
plt.title('Distribution of Candidate Stage',
          fontsize=16,
          fontweight='bold',
          pad=15)

# Axis Labels
plt.xlabel('Candidate Stage',
           fontsize=13,
           fontweight='bold')

plt.ylabel('Number of Candidates',
           fontsize=13,
           fontweight='bold')

# Rotate x-axis labels
plt.xticks(rotation=45,
           ha='right',
           fontsize=11)

plt.yticks(fontsize=11)

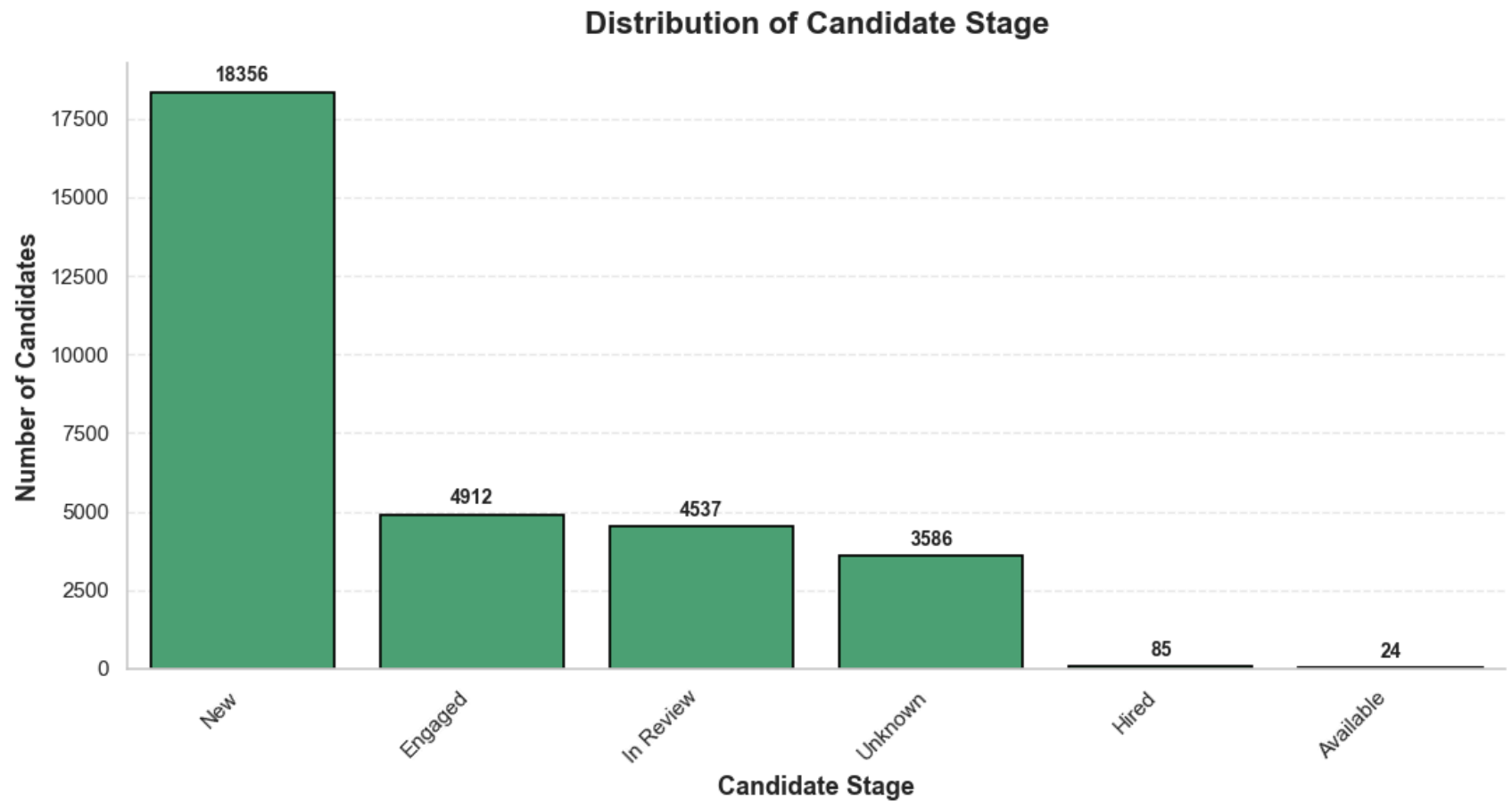
# Add values on top of bars
for container in ax.containers:
    ax.bar_label(container,
                 fontsize=10,
                 fontweight='bold',
                 padding=3)

```

```
# Remove unnecessary borders
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

# Light horizontal grid
plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()
```



```
In [33]: #Licensing Status
df['Licensing Status'].value_counts()
```

```
Out[33]: Licensing Status
Authorized To Sell      13392
Unknown                 12215
Expired                 4406
Not Authorized To Sell  1121
Surrendered            248
Pending                65
Suspended              38
Revoked                13
Application Refused     1
Non-Active              1
Name: count, dtype: int64
```

```
In [34]: #Bar Chart for Licensing Status
licensing_status = df['Licensing Status'].value_counts()

# Professional style
sns.set_theme(style="whitegrid")

plt.figure(figsize=(8,6))

ax = sns.barplot(
    x=licensing_status.index,
    y=licensing_status.values,
    color='darkorange',
    edgecolor='black',
    linewidth=1.2
)

# Title
plt.title('Distribution of Licensing Status',
          fontsize=16,
          fontweight='bold',
          pad=15)

# Axis Labels
plt.xlabel('Licensing Status',
           fontsize=13,
           fontweight='bold')

plt.ylabel('Number of Candidates',
           fontsize=13,
```

```
        fontweight='bold')

# Rotate x-axis labels
plt.xticks(rotation=45,
            ha='right',
            fontsize=11)

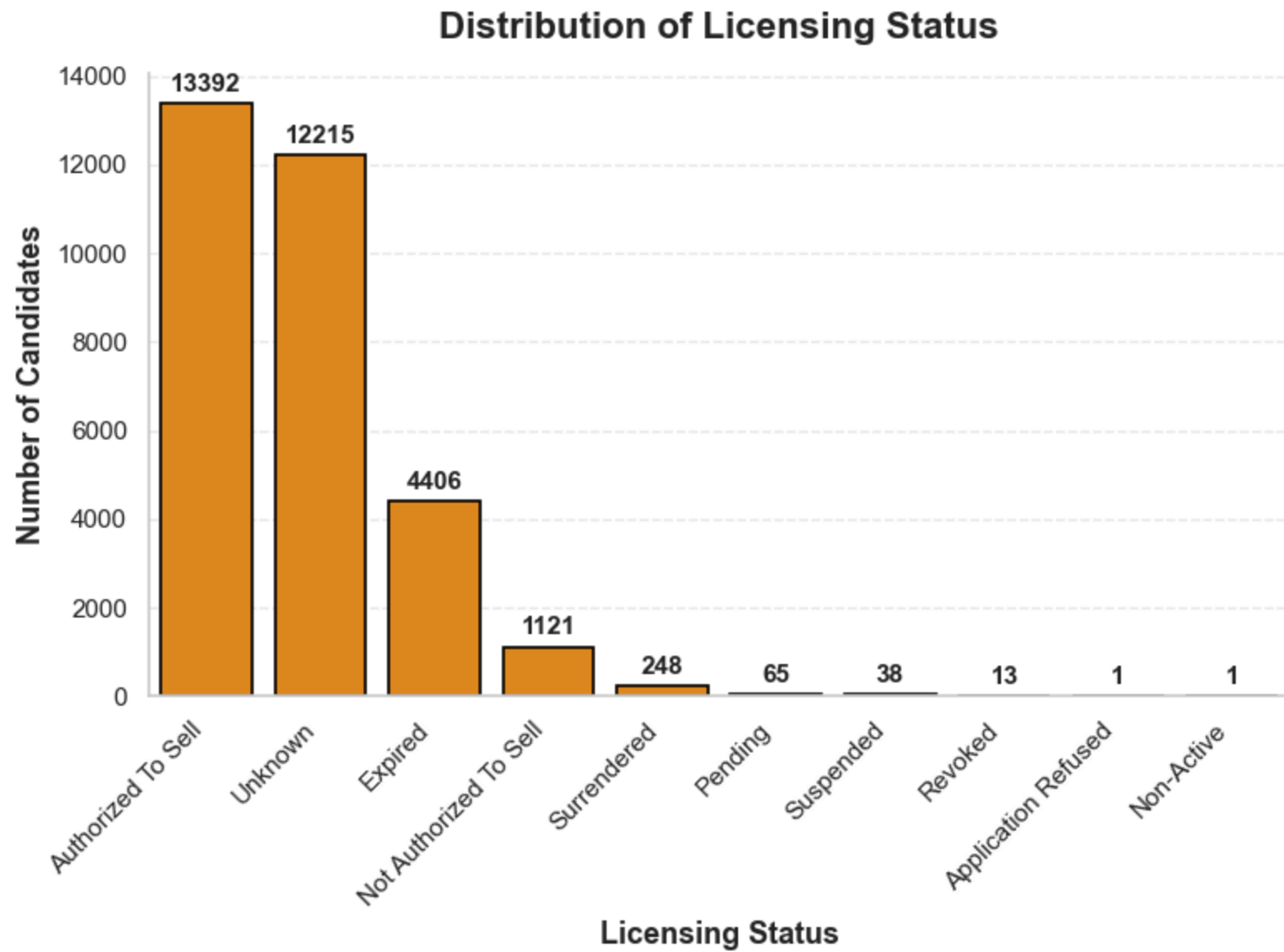
# Tick formatting
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

# Add value labels
for container in ax.containers:
    ax.bar_label(container,
                 fontsize=11,
                 fontweight='bold',
                 padding=3)

# Remove unnecessary borders
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

# Add light grid
plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()
```



```
In [35]: #Province  
df['Province'].value_counts()
```

```
Out[35]: Province
Ontario          27355
On                1653
Alberta          1524
Quebec           368
Unknown          363
British Columbia 189
New Brunswick    18
Saskatchewan     10
Manitoba         7
Nova Scotia      5
Newfoundland And Labrador 3
Prince Edward Island 2
Alaska           1
Florida          1
Michigan         1
Name: count, dtype: int64
```

```
In [36]: #Bar Chart for Province
province = df['Province'].value_counts()

# Professional style
sns.set_theme(style="whitegrid")

plt.figure(figsize=(12,6))

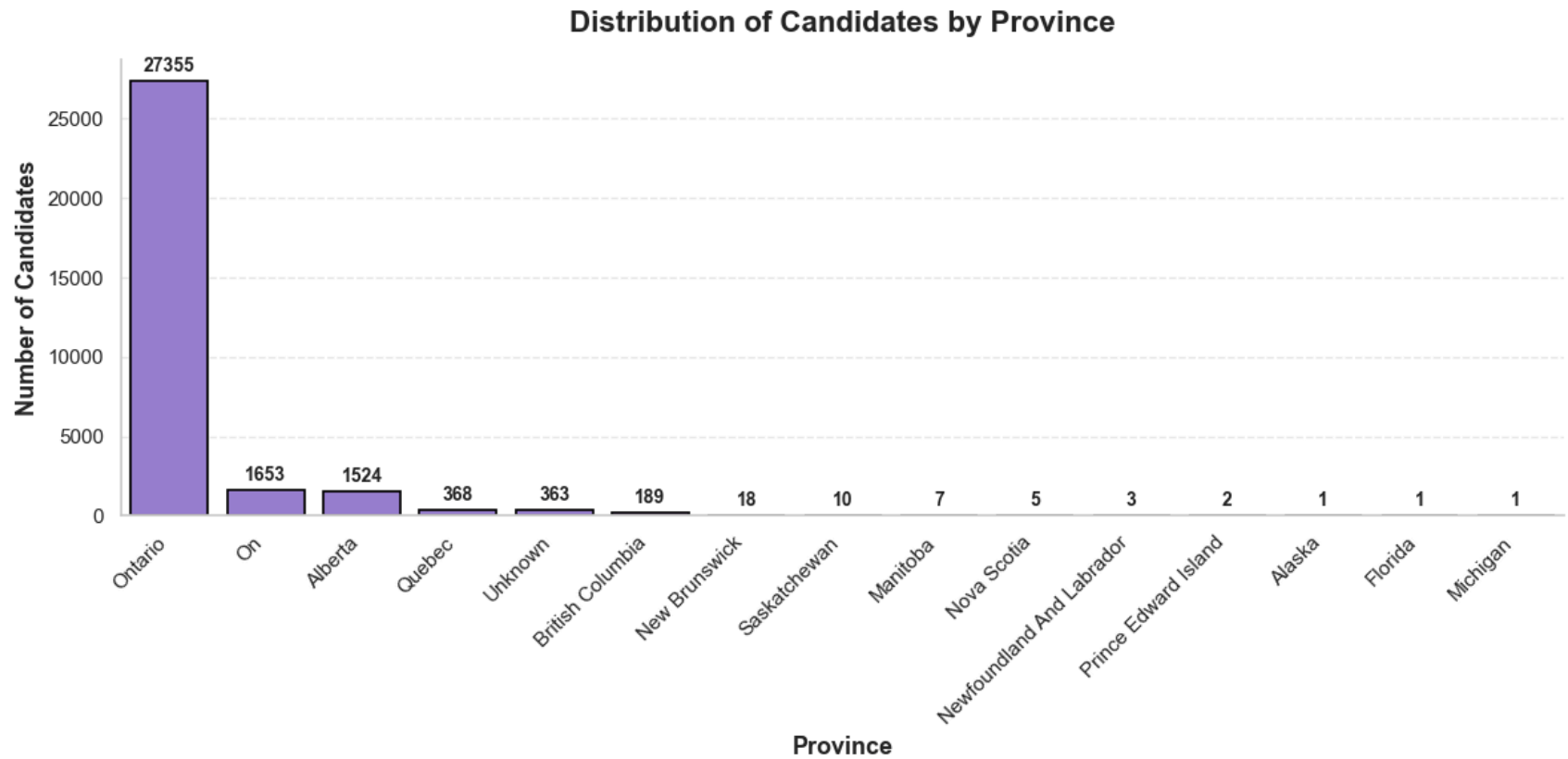
ax = sns.barplot(
    x=province.index,
    y=province.values,
    color='mediumpurple',
    edgecolor='black',
    linewidth=1.2
)

# Title
plt.title('Distribution of Candidates by Province',
          fontsize=16,
          fontweight='bold',
          pad=15)

# Axis labels
plt.xlabel('Province',
```



```
        fontsize=13,  
        fontweight='bold')  
  
plt.ylabel('Number of Candidates',  
          fontsize=13,  
          fontweight='bold')  
  
# Rotate province names  
plt.xticks(rotation=45,  
          ha='right',  
          fontsize=11)  
  
plt.yticks(fontsize=11)  
  
# Add count Labels  
for container in ax.containers:  
    ax.bar_label(container,  
                fontsize=10,  
                fontweight='bold',  
                padding=3)  
  
# Remove unnecessary borders  
ax.spines['top'].set_visible(False)  
ax.spines['right'].set_visible(False)  
  
# Add light grid  
plt.grid(axis='y',  
        linestyle='--',  
        alpha=0.4)  
  
plt.tight_layout()  
plt.show()
```



```
In [37]: #Years of Experience  
df['Years of Experience'].describe()
```

```
Out[37]: count    31500.000000  
mean         6.566032  
std          4.733863  
min          0.000000  
25%          4.000000  
50%          5.000000  
75%          7.000000  
max         18.000000  
Name: Years of Experience, dtype: float64
```

```
In [38]: # Histogram for Years of Experience  
sns.set_theme(style="whitegrid")
```

```

plt.figure(figsize=(10,6))

ax = sns.histplot(
    data=df,
    x='Years of Experience',
    bins=15,
    color='teal',
    edgecolor='black',
    linewidth=1,
    kde=True
)

# Calculate the mean
mean_exp = df['Years of Experience'].mean()

# Add a vertical line for the mean
plt.axvline(mean_exp,
            color='red',
            linestyle='--',
            linewidth=2,
            label=f'Mean = {mean_exp:.2f} years')

# Title and Labels
plt.title('Distribution of Years of Experience',
         fontsize=16,
         fontweight='bold',
         pad=15)

plt.xlabel('Years of Experience',
          fontsize=13,
          fontweight='bold')

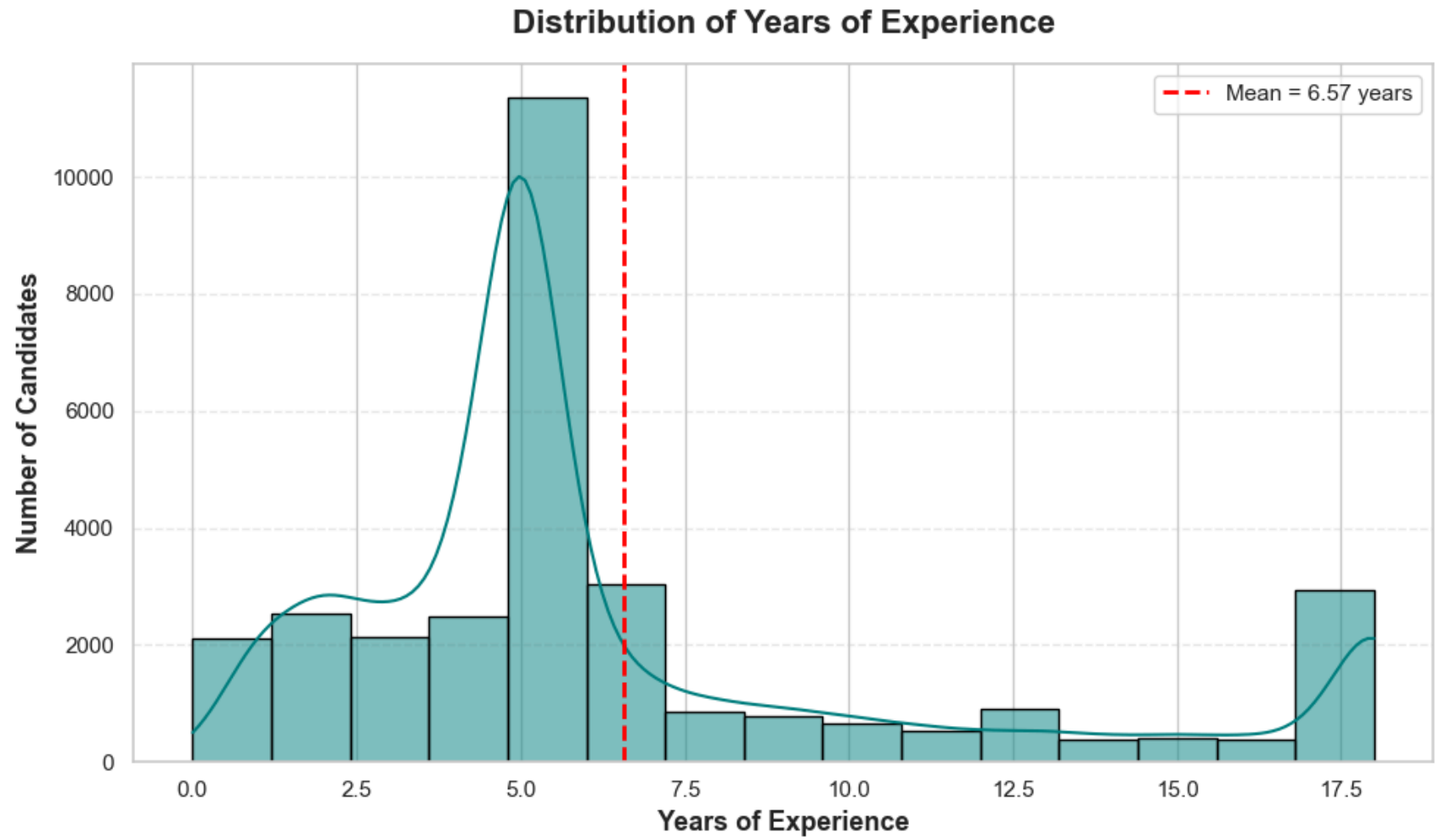
plt.ylabel('Number of Candidates',
          fontsize=13,
          fontweight='bold')

# Improve appearance
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

plt.grid(axis='y', linestyle='--', alpha=0.4)

```

```
plt.legend()  
  
plt.tight_layout()  
plt.show()
```



The descriptive analysis provided an initial overview of the 31,500 candidate records, highlighting the distribution of candidate characteristics, recruitment sources, candidate statuses and hiring outcomes. The findings showed that candidate volume was concentrated across a small number of recruitment sources, while the overall proportion progressing to validated hiring remained relatively low. These results provide a baseline understanding of the recruitment population and outcomes, supporting the subsequent exploratory and recruitment funnel analyses by identifying key patterns and areas requiring further investigation.

Funnel Analysis

```
In [39]: # Inspect the values in each relevant column

print(df['Schedule Discussion'].value_counts(dropna=False))
```

```
Schedule Discussion
No          29437
Yes         1982
Unknown      81
Name: count, dtype: int64
```

```
In [40]: print(df['Discussion Stage'].value_counts(dropna=False))
```

```
Discussion Stage
Unknown    29512
Stage 1    1982
Stage 2      6
Name: count, dtype: int64
```

```
In [41]: print(df['Candidate Status'].value_counts(dropna=False))
```

Candidate Status	
Associated	9933
New	7596
Attempted To Contact	4025
Hired	1796
Interview-Scheduled	1649
Candidate Refused To Join	1275
Following Up To Join	983
On-Hold	838
No-Show	788
Joined Another Brokerage	709
Junk Candidate	670
Non Responsive	607
Candidate Refused To Meet	455
Interview-To-Be-Scheduled	53
Rejected By Hiring Manager	50
Joined	38
Offer-Withdrawn	24
Pre-Course	6
Offer-Made	3
Unqualified	2

Name: count, dtype: int64

```
In [42]: print(df['Licensing Status'].value_counts(dropna=False))
```

Licensing Status	
Authorized To Sell	13392
Unknown	12215
Expired	4406
Not Authorized To Sell	1121
Surrendered	248
Pending	65
Suspended	38
Revoked	13
Application Refused	1
Non-Active	1

Name: count, dtype: int64

```
In [43]: print(df['Hiring Validation'].value_counts(dropna=False))
```

```
Hiring Validation
No      30042
Yes      1458
Name: count, dtype: int64
```

```
In [44]: print(df['Discussion Type'].value_counts(dropna=False))
```

```
Discussion Type
Unknown          29510
Direct Transfer   1597
Scheduled         393
Name: count, dtype: int64
```

```
In [45]: print(df['Candidate Stage'].value_counts(dropna=False))
```

```
Candidate Stage
New            18356
Engaged        4912
In Review      4537
Unknown        3586
Hired           85
Available       24
Name: count, dtype: int64
```

```
In [46]: # Funnel Table
```

```
data_frame = len(df)

contacted = df['Candidate Status'].isin(['Attempted To Contact',
                                         'Following Up To Join',
                                         'On-Hold',
                                         'Joined Another Brokerage',
                                         'Non Responsive',
                                         'Hired',
                                         'Candidate Refused To Join',
                                         'No-Show',
                                         'Candidate Refused To Meet',
                                         'Rejected By Hiring Manager'
                                         ]).sum()

discussion_scheduled = df['Candidate Status'].isin([
    'Following Up To Join',
    'On-Hold',
```

```

    'Joined Another Brokerage',
    'Non Responsive',
    'Hired',
    'Candidate Refused To Join',
    'No-Show',
    'Candidate Refused To Meet',
    'Rejected By Hiring Manager']]).sum()

discussion_completed = df['Candidate Status'].isin(['Following Up To Join',
                                                    'On-Hold',
                                                    'Joined Another Brokerage',
                                                    'Non Responsive',
                                                    'Hired',
                                                    'Candidate Refused To Join',
                                                    'Rejected By Hiring Manager'
                                                    ]).sum()

follow_up = df['Candidate Status'].isin(['Following Up To Join',
                                         'Joined Another Brokerage',
                                         'Hired',
                                         'Candidate Refused To Join'
                                         ]).sum()

validated = (df['Hiring Validation'] == 'Yes').sum()

hired = (df['Candidate Status'] == 'Hired').sum()

```

```

In [47]: funnel = {
    'Stage':[
        'Data Frame',
        'Contacted',
        'Discussion Scheduled',
        'Discussion Completed',
        'Follow Up',
        'Hiring Validation',
        'Hired'
    ],
    'Candidates':[
        data_frame,
        contacted,
        discussion_scheduled,
        discussion_completed,

```



```

        follow_up,
        validated,
        hired
    ]
}

funnel_df = pd.DataFrame(funnel)

funnel_df

```

Out[47]:

	Stage	Candidates
0	Data Frame	31500
1	Contacted	11526
2	Discussion Scheduled	7501
3	Discussion Completed	6258
4	Follow Up	4763
5	Hiring Validation	1458
6	Hired	1796

```

In [48]: # Calculate Conversion Rate
funnel_df['Conversion Rate (%)'] = (
    funnel_df['Candidates']
    / funnel_df['Candidates'].shift(1)
    *100
)

funnel_df.loc[0, 'Conversion Rate (%)']=100

funnel_df

```

Out[48]:

	Stage	Candidates	Conversion Rate (%)
0	Data Frame	31500	100.000000
1	Contacted	11526	36.590476
2	Discussion Scheduled	7501	65.078952
3	Discussion Completed	6258	83.428876
4	Follow Up	4763	76.110578
5	Hiring Validation	1458	30.610959
6	Hired	1796	123.182442

In [49]:

```
# Calculate Drop Off Rate
funnel_df['Drop-off Rate (%)'] = (
    100 - funnel_df['Conversion Rate (%)']
)

funnel_df
```

Out[49]:

	Stage	Candidates	Conversion Rate (%)	Drop-off Rate (%)
0	Data Frame	31500	100.000000	0.000000
1	Contacted	11526	36.590476	63.409524
2	Discussion Scheduled	7501	65.078952	34.921048
3	Discussion Completed	6258	83.428876	16.571124
4	Follow Up	4763	76.110578	23.889422
5	Hiring Validation	1458	30.610959	69.389041
6	Hired	1796	123.182442	-23.182442

In [50]:

```
# Overall Completion Rate
completion_rate = (
    hired/data_frame
```

```
)*100

print(f"Overall Completion Rate: {completion_rate:.2f}%")
```

Overall Completion Rate: 5.70%

In [51]: *# Bar Chart*

```
from matplotlib.ticker import FuncFormatter

sns.set_theme(style="whitegrid", font_scale=1.1)

plt.figure(figsize=(10,7))

colors = sns.color_palette("crest", len(funnel_df))

ax = sns.barplot(
    data=funnel_df,
    y='Stage',
    x='Candidates',
    palette=colors,
    edgecolor='black',
    linewidth=1.2
)

plt.title('Recruitment Funnel Analysis',
          fontsize=18,
          fontweight='bold',
          pad=20)

plt.xlabel('Number of Candidates',
           fontsize=14,
           fontweight='bold')

plt.ylabel('Recruitment Stage',
           fontsize=14,
           fontweight='bold')

ax.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{int(x):,}'))

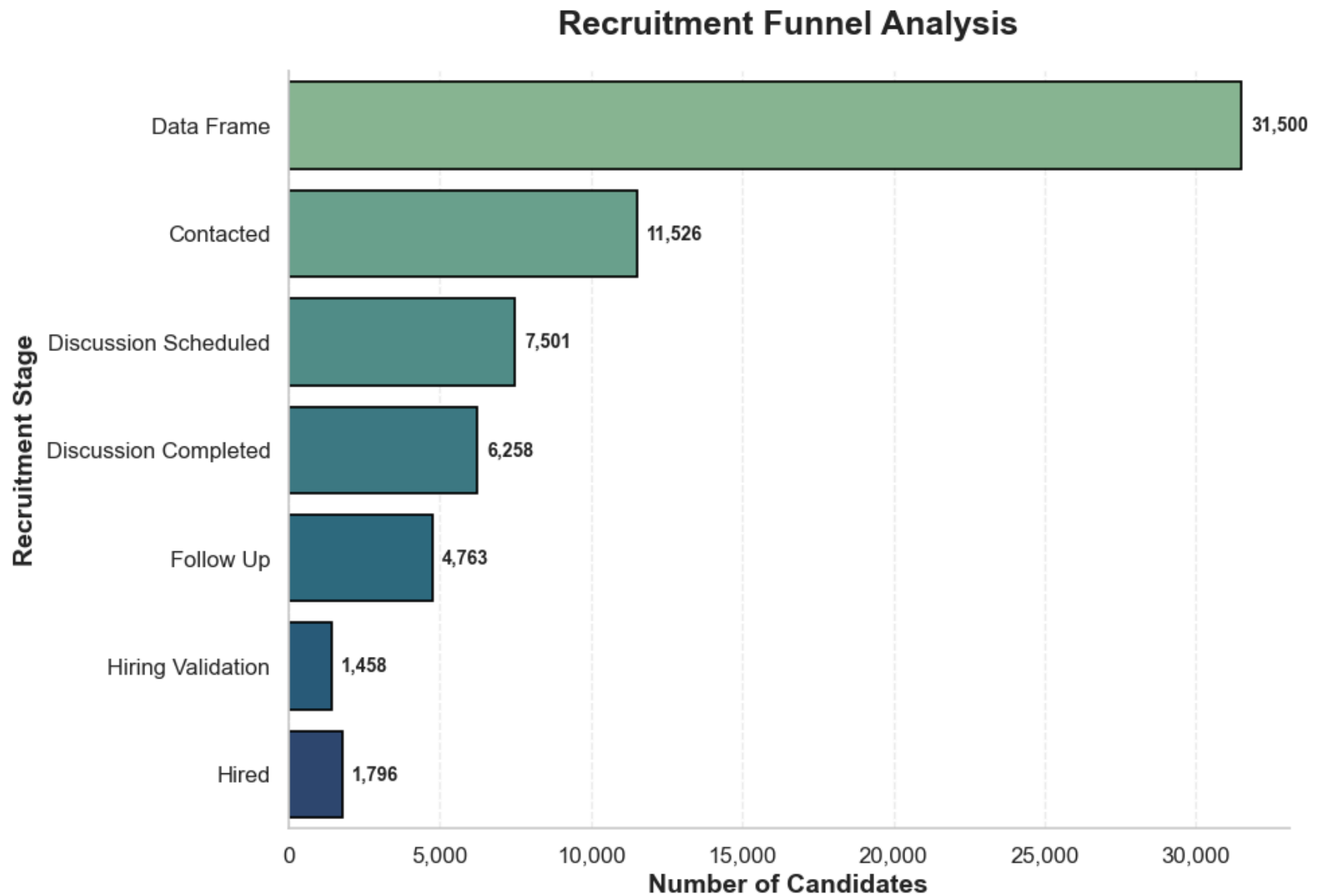
for container in ax.containers:
    labels = [f'{int(v):,}' for v in container.datavalues]
```

```
ax.bar_label(
    container,
    labels=labels,
    padding=5,
    fontsize=10,
    fontweight='bold'
)

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.grid(axis='x', linestyle='--', alpha=0.35)

plt.tight_layout()
plt.show()
```



```
In [52]: # Funnel Chart
import plotly.graph_objects as go

fig = go.Figure(go.Funnel(
    y=funnel_df['Stage'],
    x=funnel_df['Candidates'],
```

```

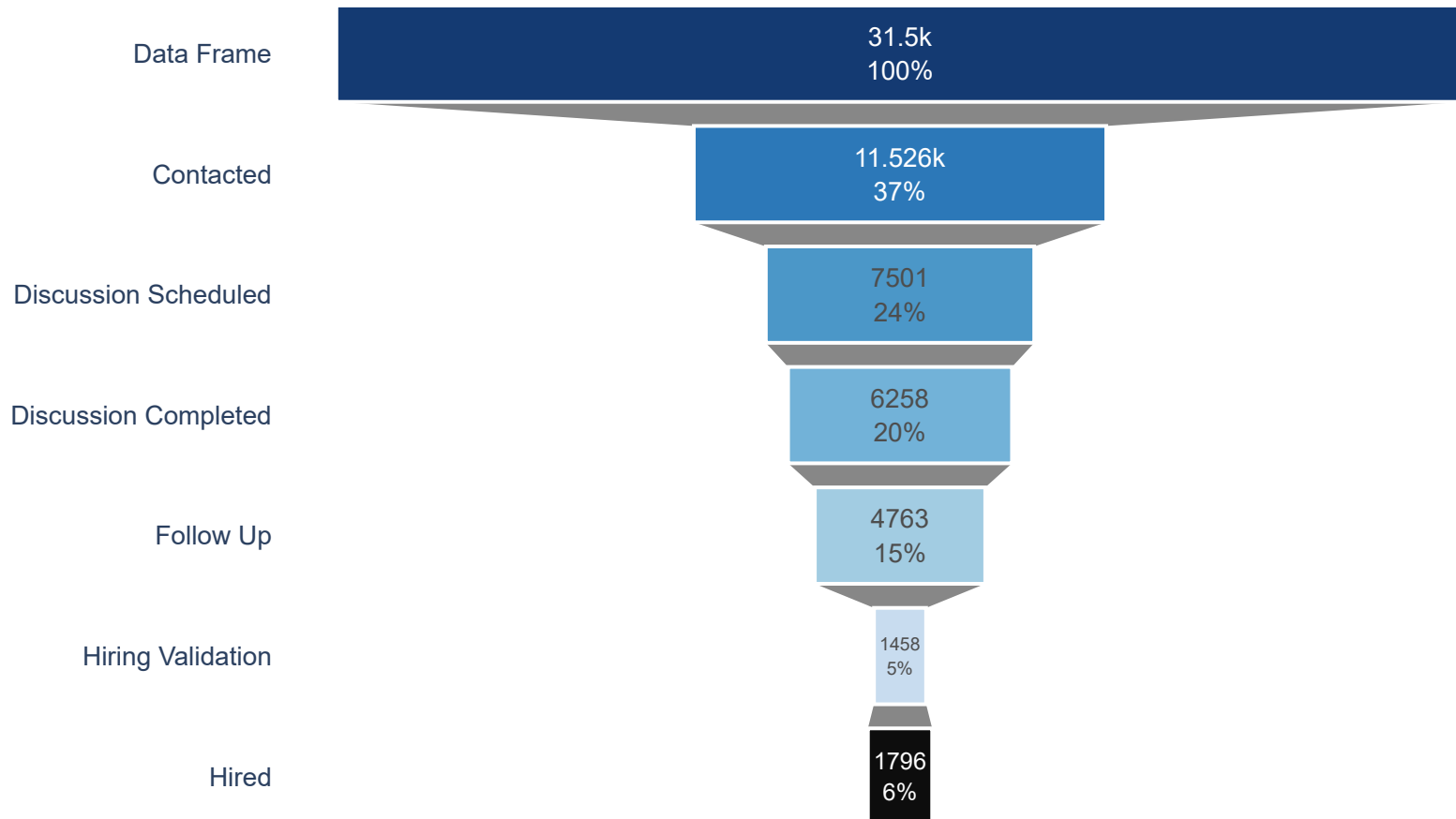
textinfo='value+percent initial',
textposition='inside',
marker=dict(
    color=[
        '#08306B',
        '#2171B5',
        '#4292C6',
        '#6BAED6',
        '#9ECAE1',
        '#C6DBEF'
    ],
    line=dict(color='white', width=2)
),
opacity=0.95
))

fig.update_layout(
    title={
        'text': '<b>Recruitment Funnel Analysis</b>',
        'x':0.5,
        'font':dict(size=24)
    },
    font=dict(
        family='Arial',
        size=14
    ),
    plot_bgcolor='white',
    paper_bgcolor='white',
    margin=dict(l=80, r=80, t=90, b=60),
    width=900,
    height=600
)

fig.show()

```

Recruitment Funnel Analysis



The recruitment funnel analysis demonstrated a clear pattern of candidate attrition across the recruitment process. From the initial 31,500 candidates, 11,526 were contacted, 7,501 reached discussion scheduling, 6,258 completed discussions, and 4,763 progressed to follow-up. The most significant bottleneck occurred between Follow Up and Hiring Validation, where only 1,458 candidates

progressed, representing a 30.61% conversion rate and 69.39% drop-off. However, the final stage shows an increase from 1,458 candidates at Hiring Validation to 1,796 candidates recorded as Hired. This increase should be interpreted in the context of the data history, as Hiring Validation was introduced after 2023 and was not consistently maintained in earlier records. Consequently, some candidates who were recorded as Hired may not have a corresponding Hiring Validation record, resulting in the apparent increase at the final stage. Therefore, the funnel highlights both a significant recruitment bottleneck and an important historical data-recording limitation that is considered.

Exploratory Data Analysis (EDA)

```
In [53]: # Experience vs Hiring Validation

sns.set_theme(style="whitegrid")

plt.figure(figsize=(8,6))

ax = sns.boxplot(
    data=df,
    x='Hiring Validation',
    y='Years of Experience',
    palette='Set2'
)

plt.title('Years of Experience by Hiring Validation',
          fontsize=16,
          fontweight='bold')

plt.xlabel('Hiring Validation',
           fontsize=13,
           fontweight='bold')

plt.ylabel('Years of Experience',
           fontsize=13,
           fontweight='bold')

plt.tight_layout()

plt.show()
```




```
In [54]: # Licensing Status vs Hiring Validation
```

```
licensing_hiring = pd.crosstab(  
    df['Licensing Status'],  
    df['Hiring Validation']  
)
```

```
licensing_hiring
```

Out[54]:

Hiring Validation	No	Yes
Licensing Status		
Application Refused	1	0
Authorized To Sell	12871	521
Expired	3979	427
Non-Active	0	1
Not Authorized To Sell	1023	98
Pending	62	3
Revoked	13	0
Surrendered	235	13
Suspended	38	0
Unknown	11820	395

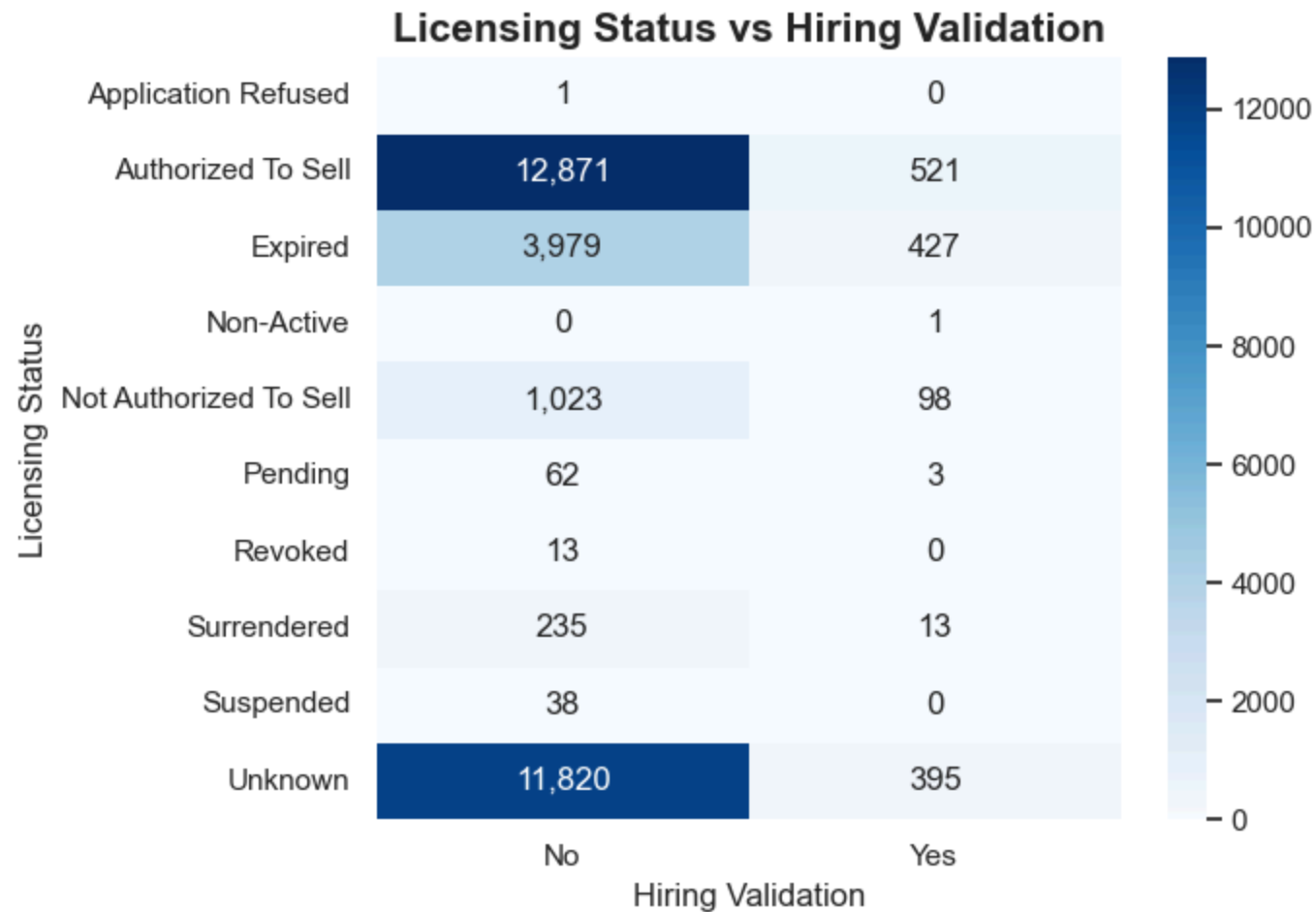
```
In [55]: # Heatmap of Licensing Status vs Hiring Validation
plt.figure(figsize=(7,5))

sns.heatmap(
    licensing_hiring,
    annot=True,
    fmt=',',
    cmap='Blues'
)

plt.title('Licensing Status vs Hiring Validation',
          fontsize=15,
          fontweight='bold')

plt.tight_layout()
```

```
plt.show()
```



```
In [56]: # Hiring Validation vs Candidate Status
validation_status = pd.crosstab(
    df['Hiring Validation'],
    df['Candidate Status']
)

validation_status
```

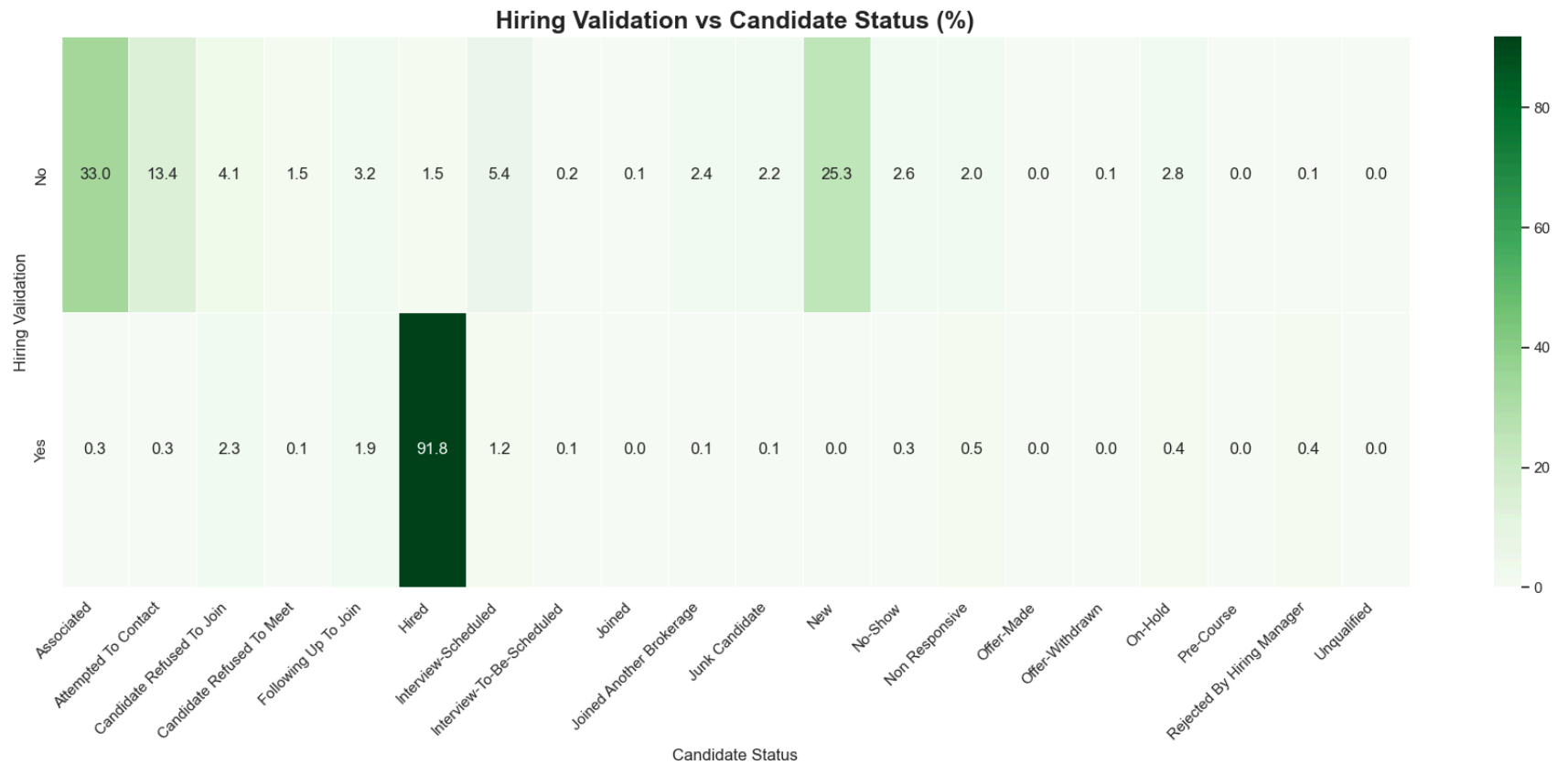
Out[56]:

	Candidate Status	Associated	Attempted To Contact	Candidate Refused To Join	Candidate Refused To Meet	Following Up To Join	Hired	Interview-Scheduled	Interview-To-Be-Scheduled	Joined	Joined Another Brokerage	Joined Another Company
Hiring Validation												
No		9928	4020	1241	454	955	457	1631	52	38	708	6
Yes		5	5	34	1	28	1339	18	1	0	1	



In [57]: *# Heatmap for Hiring Validation vs Candidate Status*

```
discussion_percent = pd.crosstab(  
    df['Hiring Validation'],  
    df['Candidate Status'],  
    normalize='index'  
) * 100  
  
plt.figure(figsize=(18,8))  
  
sns.heatmap(  
    discussion_percent,  
    annot=True,  
    fmt='.1f',  
    cmap='Greens',  
    linewidths=0.5  
)  
  
plt.title('Hiring Validation vs Candidate Status (%)',  
          fontsize=18,  
          fontweight='bold')  
  
plt.xticks(rotation=45, ha='right')  
  
plt.tight_layout()  
plt.show()
```



```
In [58]: # Live Transfer vs Hiring Validation
live_transfer = pd.crosstab(
    df['Live Transfer'],
    df['Hiring Validation']
)

live_transfer
```

Out[58]: **Hiring Validation** **No** **Yes**

Live Transfer		
Not Offered	5	0
Offered	1657	327
Unknown	28380	1131

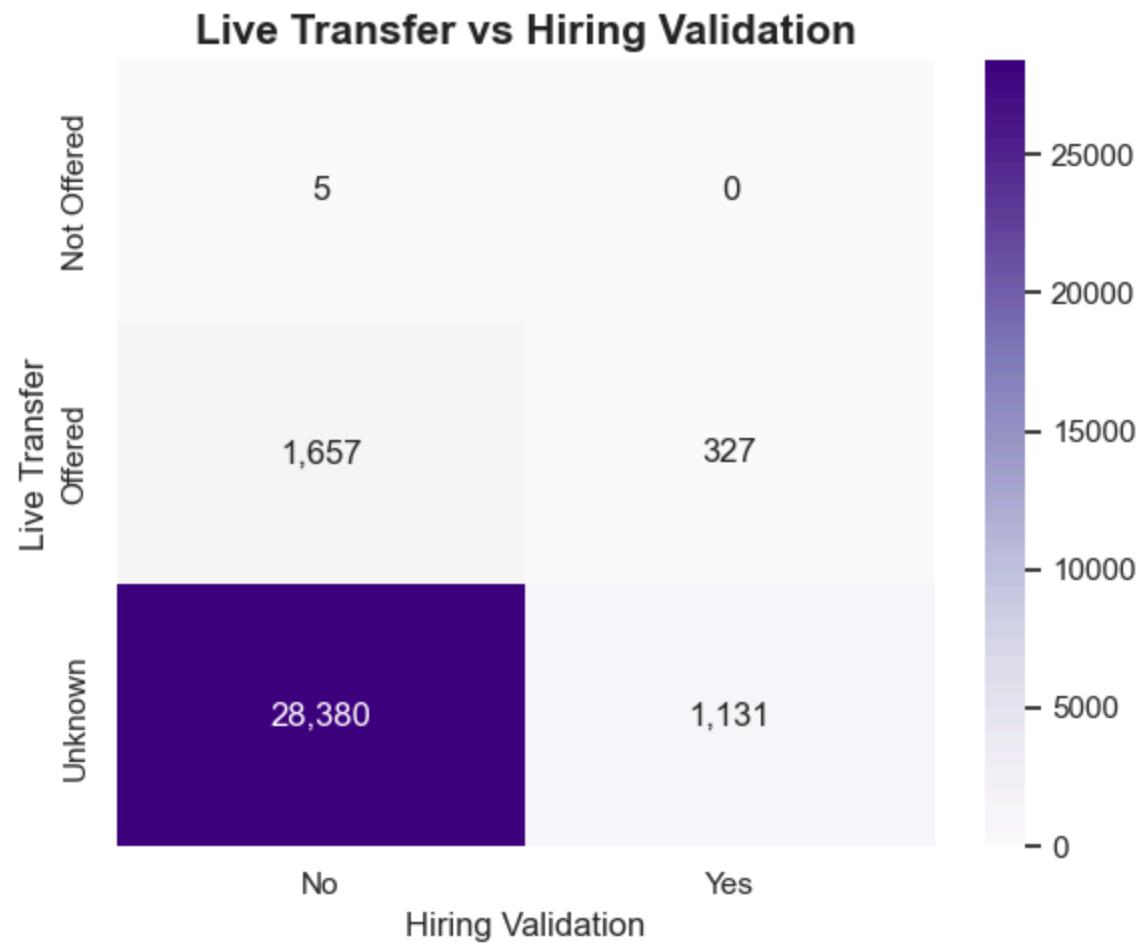
```
In [59]: # Heatmap for Live Transfer vs Hiring Validation
plt.figure(figsize=(6,5))

sns.heatmap(
    live_transfer,
    annot=True,
    fmt=',',
    cmap='Purples'
)

plt.title('Live Transfer vs Hiring Validation',
          fontsize=15,
          fontweight='bold')

plt.tight_layout()

plt.show()
```



```
In [60]: # Correlation Analysis  
  
correlation = df.corr(numeric_only=True)
```

```
In [61]: # Heatmap of Correlation  
  
plt.figure(figsize=(14,10))  
  
sns.heatmap(  
    correlation,  
    annot=True,  
    fmt=".2f",
```

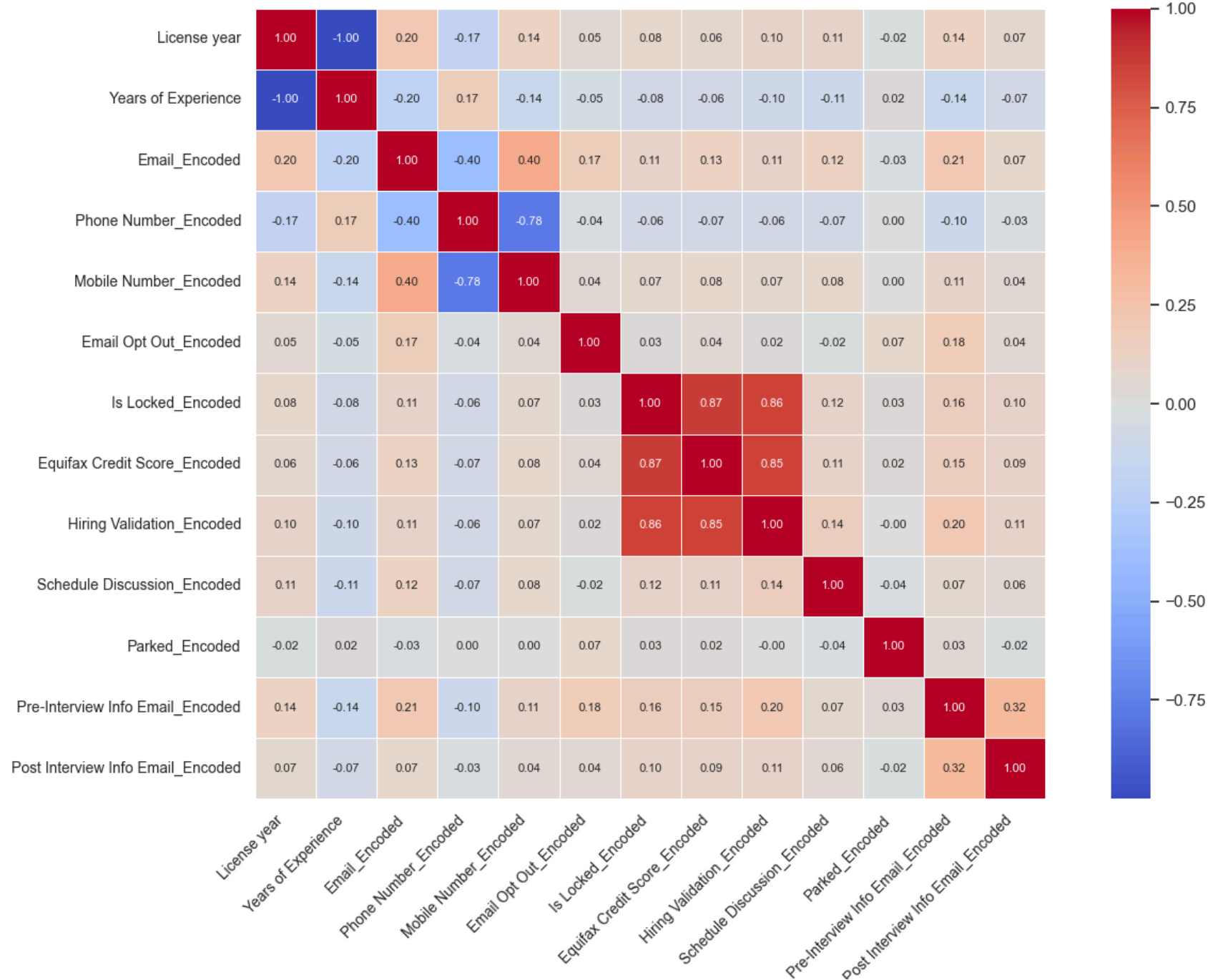
```
        cmap="coolwarm",
        center=0,
        linewidths=0.5,
        annot_kws={"size":8},
        square=True
    )

plt.title(
    "Correlation Matrix",
    fontsize=18,
    fontweight="bold",
    pad=20
)

plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(rotation=0, fontsize=10)

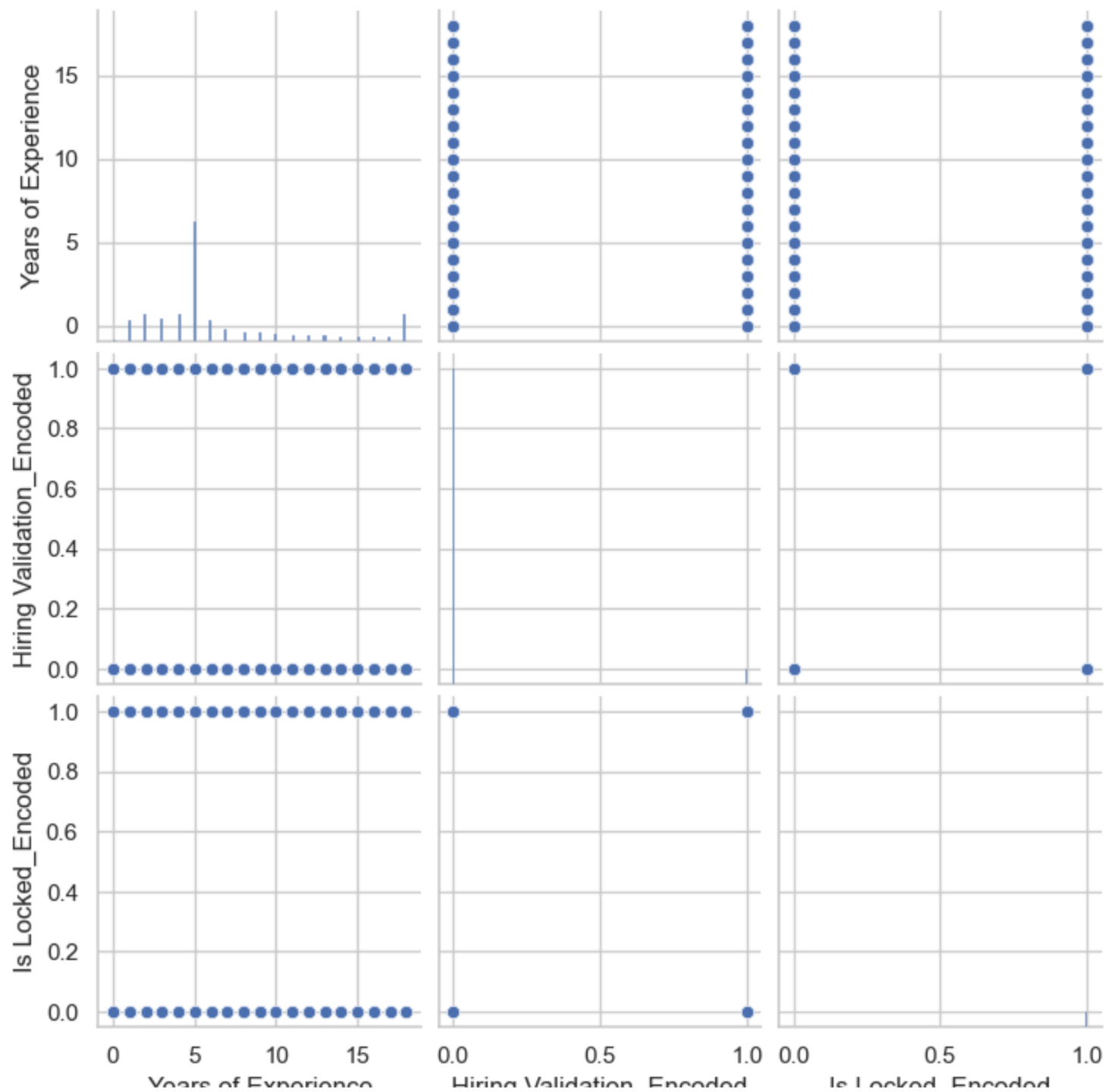
plt.tight_layout()
plt.show()
```


Correlation Matrix



```
In [62]: sns.pairplot(
    df[
        [
            'Years of Experience',
            'Hiring Validation_Encoded',
            'Is Locked_Encoded'
        ]
    ]
)

plt.show()
```



Years of Experience

Hiring Validation_Encoded

Is Locked_Encoded

Overall, the EDA identified several important relationships and patterns within the recruitment data, particularly around Hiring Validation, licensing status, candidate status, years of experience, and live-transfer activity. The analysis indicated that candidates offered a live transfer had a substantially higher validation rate (16.48%) compared with those who were not offered a live transfer (3.83%), while licensing status also showed variation in validation outcomes. These findings provide useful evidence of factors associated with candidate progression; however, they should be interpreted as observed associations rather than causal relationships, as the analysis does not establish that these factors directly cause successful recruitment outcomes.

Feature Engineering

```
In [63]: # Duplicating the Dataset  
df_m1 = df.copy()
```

```
In [64]: df_m1.head()
```

Out[64]:

	Candidate ID.1	Gender	Email	Phone Number	Mobile Number	City	Country	Created Time	Modified Time	Currency	...	Phone Number_Encoded
0	Zr_28480_Cand	Male	Yes	No	Yes	North York	Canada	2023-12-29 09:31:00	2026-03-06 13:01:00	Cad	...	0.0
1	Zr_28476_Cand	Female	Yes	Yes	No	Cambridge	Canada	2023-12-20 16:31:00	2026-05-06 14:43:00	Cad	...	1.0
2	Zr_28475_Cand	Male	Yes	No	Yes	Brampton	Canada	2023-12-20 16:29:00	2026-05-06 14:43:00	Cad	...	0.0
3	Zr_28474_Cand	Male	Yes	No	Yes	Toronto	Canada	2023-12-19 08:54:00	2026-05-06 14:43:00	Cad	...	0.0
4	Zr_28473_Cand	Male	Yes	No	Yes	Cambridge	Canada	2023-12-19 08:54:00	2026-05-06 14:43:00	Cad	...	0.0

5 rows × 56 columns



```
In [65]: df_ml['Live Transfer'] = df_ml['Live Transfer'].map({
    'Offered':1,
    'Not Offered':0,
    'Unknown':0
})
```

```
In [66]: df_ml['Gender'] = df_ml['Gender'].map({
    'Male':1,
    'Female':0
})
```

```
In [67]: encoder = LabelEncoder()
```

```
In [68]: categorical_columns = [  
    'Province',  
    'Candidate Status',  
    'Candidate Stage',  
    'Discussion Stage',  
    'Licensing Status',  
    'Source',  
    'Ethnicity',  
    'Religion',  
    'Position Category',  
    'License Class',  
    'Current and Previous Mortgage',  
    'Hiring Validation'  
]
```

```
In [69]: for col in categorical_columns:  
    df_ml[col] = encoder.fit_transform(df_ml[col].astype(str))
```

```
In [70]: df_ml = df_ml.drop(columns=[  
    'Candidate ID.1',  
    'Email',  
    'Phone Number',  
    'Mobile Number',  
    'City',  
    'Country',  
    'Currency',  
    'Last Activity Time',  
    'Last emailed',  
    'Email Opt Out',  
    'Is Hot Candidate',  
    'Is Locked',  
    'Institute 1 Sign Up Date',  
    'Equifax Credit Score',  
    'Criminal History',  
    'Prior Discipline Action by a Regulator',  
    'Professional License Suspension',  
    '1 Month Fee Waiver',  
    'Work Type',  
    'Realtor License',  
    'Discussion Type',  
    'Schedule Discussion',  
])
```

```
'Parked',  
'Pre-Interview Info Email',  
'Post Interview Info Email',  
'Created Time',  
'Modified Time',  
'Initial Contact Date',  
'Hire Date',  
'Is Locked_Encoded',  
'Hiring Validation_Encoded'  
])
```

```
In [71]: df_ml.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 31500 entries, 0 to 31499
Data columns (total 25 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Gender                                     31500 non-null  int64
1   Source                                     31500 non-null  int64
2   Candidate Status                         31500 non-null  int64
3   Ethnicity                                31500 non-null  int64
4   Religion                                  31500 non-null  int64
5   Position Category                       31500 non-null  int64
6   License Class                           31500 non-null  int64
7   License year                             31500 non-null  float64
8   Years of Experience                     31500 non-null  float64
9   Current and Previous Mortgage           31500 non-null  int64
10  Candidate Stage                         31500 non-null  int64
11  Hiring Validation                       31500 non-null  int64
12  Licensing Status                       31500 non-null  int64
13  Province                                31500 non-null  int64
14  Live Transfer                           31500 non-null  int64
15  Discussion Stage                       31500 non-null  int64
16  Email_Encoded                           31500 non-null  float64
17  Phone Number_Encoded                   31500 non-null  float64
18  Mobile Number_Encoded                  31500 non-null  float64
19  Email Opt Out_Encoded                  31500 non-null  float64
20  Equifax Credit Score_Encoded           31500 non-null  float64
21  Schedule Discussion_Encoded             31500 non-null  float64
22  Parked_Encoded                         31500 non-null  float64
23  Pre-Interview Info Email_Encoded        31500 non-null  float64
24  Post Interview Info Email_Encoded        31500 non-null  float64
dtypes: float64(11), int64(14)
memory usage: 6.0 MB

```

```

In [72]: Y = df_ml['Hiring Validation']
X = df_ml.drop('Hiring Validation', axis=1)
print(X.head())
print(Y.head())

```


	Gender	Source	Candidate	Status	Ethnicity	Religion	Position	Category \
0	1	16		5	11	4		2
1	0	7		9	3	2		2
2	1	7		9	11	8		2
3	1	7		12	11	2		2
4	1	7		12	11	5		2

	License	Class	License	year	Years	of Experience \
0		5		2024.0		2.0
1		1		2024.0		2.0
2		5		2021.0		5.0
3		5		2021.0		5.0
4		1		2024.0		2.0

	Current	and Previous	Mortgage	...	Discussion	Stage	Email_Encoded \
0			1886	...		2	1.0
1			1788	...		2	1.0
2			1788	...		2	1.0
3			1788	...		2	1.0
4			1788	...		2	1.0

	Phone	Number_Encoded	Mobile	Number_Encoded	Email	Opt Out_Encoded \
0		0.0		1.0		0.0
1		1.0		0.0		1.0
2		0.0		1.0		1.0
3		0.0		1.0		0.0
4		0.0		1.0		1.0

	Equifax	Credit	Score_Encoded	Schedule	Discussion_Encoded	Parked_Encoded \
0			1.0		0.0	0.0
1			0.0		0.0	0.0
2			0.0		0.0	0.0
3			0.0		0.0	0.0
4			0.0		0.0	0.0

	Pre-Interview	Info	Email_Encoded	Post	Interview	Info	Email_Encoded
0			0.0				0.0
1			1.0				1.0
2			1.0				0.0
3			1.0				1.0
4			1.0				0.0

```
[5 rows x 24 columns]
0    1
1    0
2    0
3    0
4    0
Name: Hiring Validation, dtype: int64
```

```
In [73]: y = df_ml['Hiring Validation']
X_original = df_ml.drop('Hiring Validation', axis=1)
print(f"\nOriginal feature set: {X_original.shape[1]} features")
print(f"Target distribution -> Yes: {y.sum():,} ({y.mean()*100:.2f}%), "
      f"No: {(len(y)-y.sum()):,} ({(1-y.mean())*100:.2f}%)")
```

Original feature set: 24 features

Target distribution -> Yes: 1,458 (4.63%), No: 30,042 (95.37%)

Following feature engineering, the dataset was transformed into a modelling-ready structure by selecting relevant candidate-level variables and converting categorical and binary information into numerical representations. A total of 24 features were retained as predictors, with Hiring Validation defined as the target variable, consisting of 1,458 positive cases (4.63%) and 30,042 negative cases (95.37%). The engineered dataset therefore provided a consistent numerical feature set suitable for the subsequent Logistic Regression, Decision Tree, and Random Forest models, while maintaining the underlying recruitment characteristics captured in the original data

Predictive Analysis

```
In [74]: # Splitting the Dataset

X_train, X_test, y_train, y_test = train_test_split(
    X_original, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

cw = 'balanced'
```

```
In [75]: print(X_train.shape)
print(X_test.shape)
```

```
(25200, 24)
```

```
(6300, 24)
```

```
In [76]: # Logistic Regression
```

```
model = LogisticRegression(max_iter=1000, class_weight=cw)
model.fit(X_train_scaled, y_train)
model_pred = model.predict(X_test_scaled)
model_proba = model.predict_proba(X_test_scaled)[: , 1]

print(classification_report(y_test, model_pred))
print("Confusion matrix:")
print(confusion_matrix(y_test, model_pred))
print(f"PR-AUC: {average_precision_score(y_test, model_proba):.4f}")

lr_original_f1 = f1_score(y_test, model_pred, pos_label=1)
lr_original_acc = accuracy_score(y_test, model_pred)
```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	6008
1	0.77	0.99	0.86	292
accuracy			0.99	6300
macro avg	0.88	0.99	0.93	6300
weighted avg	0.99	0.99	0.99	6300

```
Confusion matrix:
[[5920  88]
 [   4 288]]
PR-AUC: 0.9643
```

```
In [77]: # Decision Tree
```

```
tree = DecisionTreeClassifier(random_state=RANDOM_STATE, class_weight=cw, ccp_alpha=0.0005)
tree.fit(X_train, y_train)
tree_pred = tree.predict(X_test)

print(classification_report(y_test, tree_pred))
```

```

print("Confusion matrix:")
print(confusion_matrix(y_test, tree_pred))

dt_original_f1 = f1_score(y_test, tree_pred, pos_label=1)
dt_original_acc = accuracy_score(y_test, tree_pred)

```

	precision	recall	f1-score	support
0	1.00	0.99	1.00	6008
1	0.90	0.99	0.94	292
accuracy			0.99	6300
macro avg	0.95	0.99	0.97	6300
weighted avg	0.99	0.99	0.99	6300

```

Confusion matrix:
[[5975  33]
 [   3 289]]

```

In [78]: *# Random Forest*

```

rf = RandomForestClassifier(random_state=RANDOM_STATE, class_weight=cw, n_estimators=300)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
rf_proba = rf.predict_proba(X_test)[:, 1]

print(classification_report(y_test, rf_pred))
print("Confusion matrix:")
print(confusion_matrix(y_test, rf_pred))
print(f"PR-AUC: {average_precision_score(y_test, rf_proba):.4f}")

rf_original_f1 = f1_score(y_test, rf_pred, pos_label=1)
rf_original_acc = accuracy_score(y_test, rf_pred)

importance_original = pd.DataFrame({
    'Feature': X_original.columns, 'Importance': rf.feature_importances_
}).sort_values('Importance', ascending=False)
print(importance_original.head(10))

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6008
1	0.97	0.97	0.97	292
accuracy			1.00	6300
macro avg	0.98	0.98	0.98	6300
weighted avg	1.00	1.00	1.00	6300

Confusion matrix:

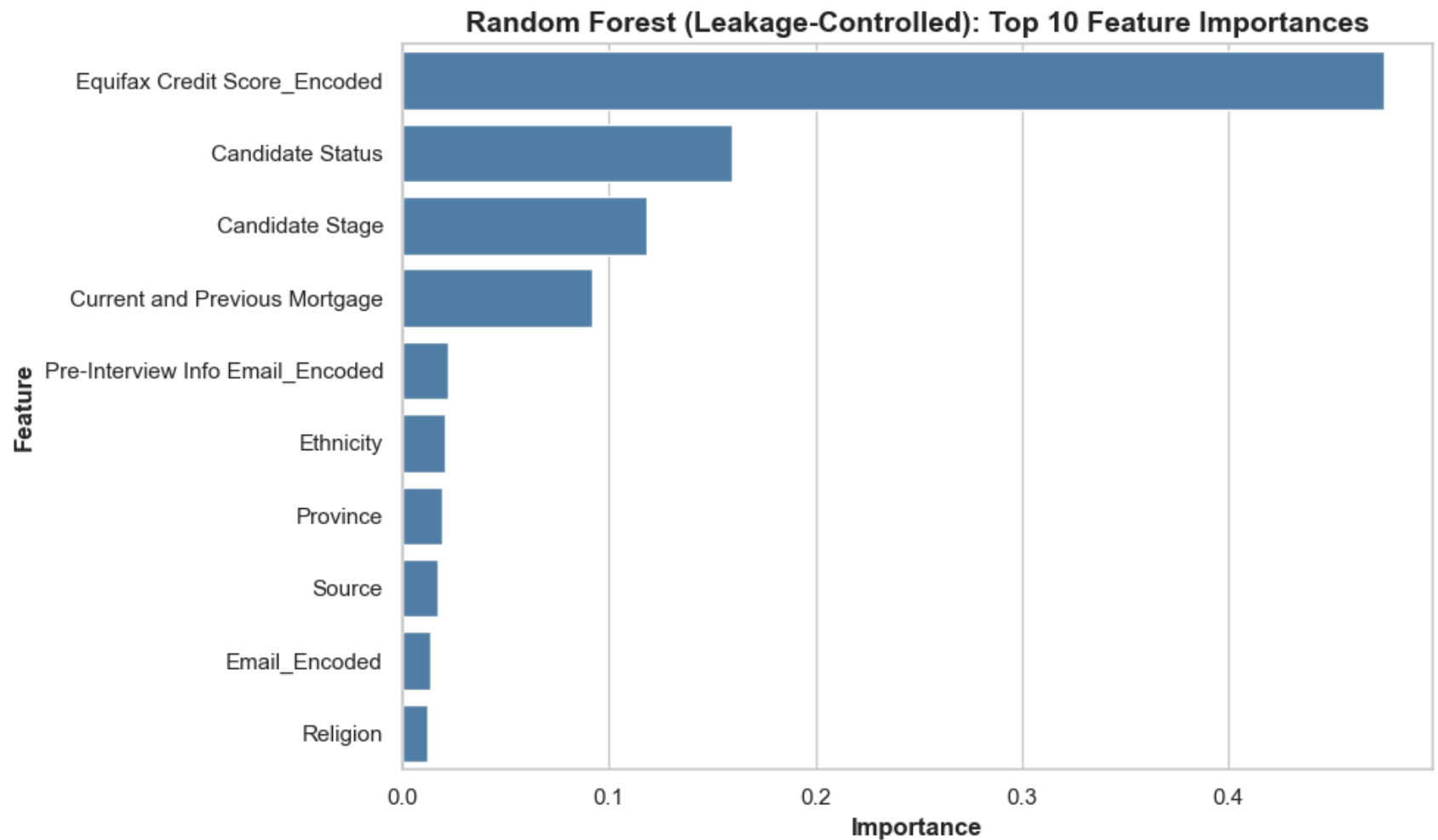
```
[[5998  10]
 [   9 283]]
```

PR-AUC: 0.9972

	Feature	Importance
19	Equifax Credit Score_Encoded	0.475356
2	Candidate Status	0.159813
10	Candidate Stage	0.117935
9	Current and Previous Mortgage	0.092082
22	Pre-Interview Info Email_Encoded	0.022249
3	Ethnicity	0.020762
12	Province	0.019173
1	Source	0.017029
15	Email_Encoded	0.013187
4	Religion	0.011946

In [79]: *# Plotting Top 10 Importance*

```
plt.figure(figsize=(10,6))
sns.barplot(
    data=importance_original.head(10),
    x='Importance',
    y='Feature',
    color='steelblue'
)
plt.title('Random Forest (Leakage-Controlled): Top 10 Feature Importances',
          fontsize=14, fontweight='bold')
plt.xlabel('Importance', fontsize=12, fontweight='bold')
plt.ylabel('Feature', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('feature_importance_leakage_controlled.png', dpi=150)
plt.show()
```



Leakage Controlled

```
In [80]: leak_risk_cols = ['Candidate Status', 'Candidate Stage', 'Equifax Credit Score_Encoded']
fairness_risk_cols = ['Ethnicity', 'Religion']
redundant_cols = ['License year'] # perfectly correlated with Years of Experience

cols_to_drop = [c for c in (leak_risk_cols + fairness_risk_cols + redundant_cols) if c in X_original.columns]
X_clean = X_original.drop(columns=cols_to_drop)

RANDOM_STATE = 42
```

```
print(f"Leakage-controlled feature set: {X_clean.shape[1]} features "  
      f"(removed: {'', ' '.join(cols_to_drop)}))")
```

Leakage-controlled feature set: 18 features (removed: Candidate Status, Candidate Stage, Equifax Credit Score_Encode
d, Ethnicity, Religion, License year)

Predictive Analysis

In [81]: *# Splitting the Dataset*

```
X_train2, X_test2, y_train2, y_test2 = train_test_split(  
    X_clean, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y  
)  
scaler2 = StandardScaler()  
X_train2_scaled = scaler2.fit_transform(X_train2)  
X_test2_scaled = scaler2.transform(X_test2)  
  
cw = 'balanced'
```

In [82]: *# Logistic Regression*

```
lr = LogisticRegression(max_iter=1000, class_weight=cw)  
lr.fit(X_train2_scaled, y_train2)  
lr_pred = lr.predict(X_test2_scaled)  
lr_proba = lr.predict_proba(X_test2_scaled)[: , 1]  
  
print(classification_report(y_test2, lr_pred))  
print("Confusion matrix:")  
print(confusion_matrix(y_test2, lr_pred))  
print(f"PR-AUC: {average_precision_score(y_test2, lr_proba):.4f}")  
  
lr_clean_f1 = f1_score(y_test2, lr_pred, pos_label=1)  
lr_clean_acc = accuracy_score(y_test2, lr_pred)
```

	precision	recall	f1-score	support
0	0.99	0.78	0.87	6008
1	0.15	0.79	0.25	292
accuracy			0.78	6300
macro avg	0.57	0.79	0.56	6300
weighted avg	0.95	0.78	0.84	6300

Confusion matrix:

```
[[4661 1347]
 [ 60 232]]
```

PR-AUC: 0.2612

In [83]: *# Decision Tree*

```
tree2 = DecisionTreeClassifier(random_state=RANDOM_STATE, class_weight=cw, ccp_alpha=0.0005)
tree2.fit(X_train2, y_train2)
tree2_pred = tree2.predict(X_test2)

print(classification_report(y_test2, tree2_pred))
print("Confusion matrix:")
print(confusion_matrix(y_test2, tree2_pred))

dt_clean_f1 = f1_score(y_test2, tree2_pred, pos_label=1)
dt_clean_acc = accuracy_score(y_test2, tree2_pred)
```

	precision	recall	f1-score	support
0	0.99	0.88	0.93	6008
1	0.26	0.87	0.40	292
accuracy			0.88	6300
macro avg	0.63	0.87	0.67	6300
weighted avg	0.96	0.88	0.91	6300

Confusion matrix:

```
[[5283 725]
 [ 39 253]]
```

In [84]: *# Random Forest*


```

rf2 = RandomForestClassifier(random_state=RANDOM_STATE, class_weight=cw, n_estimators=300)
rf2.fit(X_train2, y_train2)
rf2_pred = rf2.predict(X_test2)
rf2_proba = rf2.predict_proba(X_test2)[:, 1]

print(classification_report(y_test2, rf2_pred))
print("Confusion matrix:")
print(confusion_matrix(y_test2, rf2_pred))
print(f"PR-AUC: {average_precision_score(y_test2, rf2_proba):.4f}")

rf_clean_f1 = f1_score(y_test2, rf2_pred, pos_label=1)
rf_clean_acc = accuracy_score(y_test2, rf2_pred)

importance_clean = pd.DataFrame({
    'Feature': X_clean.columns, 'Importance': rf2.feature_importances_
}).sort_values('Importance', ascending=False)
print(importance_clean.head(10))

```

	precision	recall	f1-score	support
0	0.98	0.98	0.98	6008
1	0.56	0.61	0.58	292
accuracy			0.96	6300
macro avg	0.77	0.79	0.78	6300
weighted avg	0.96	0.96	0.96	6300

Confusion matrix:

```

[[5866 142]
 [ 113 179]]

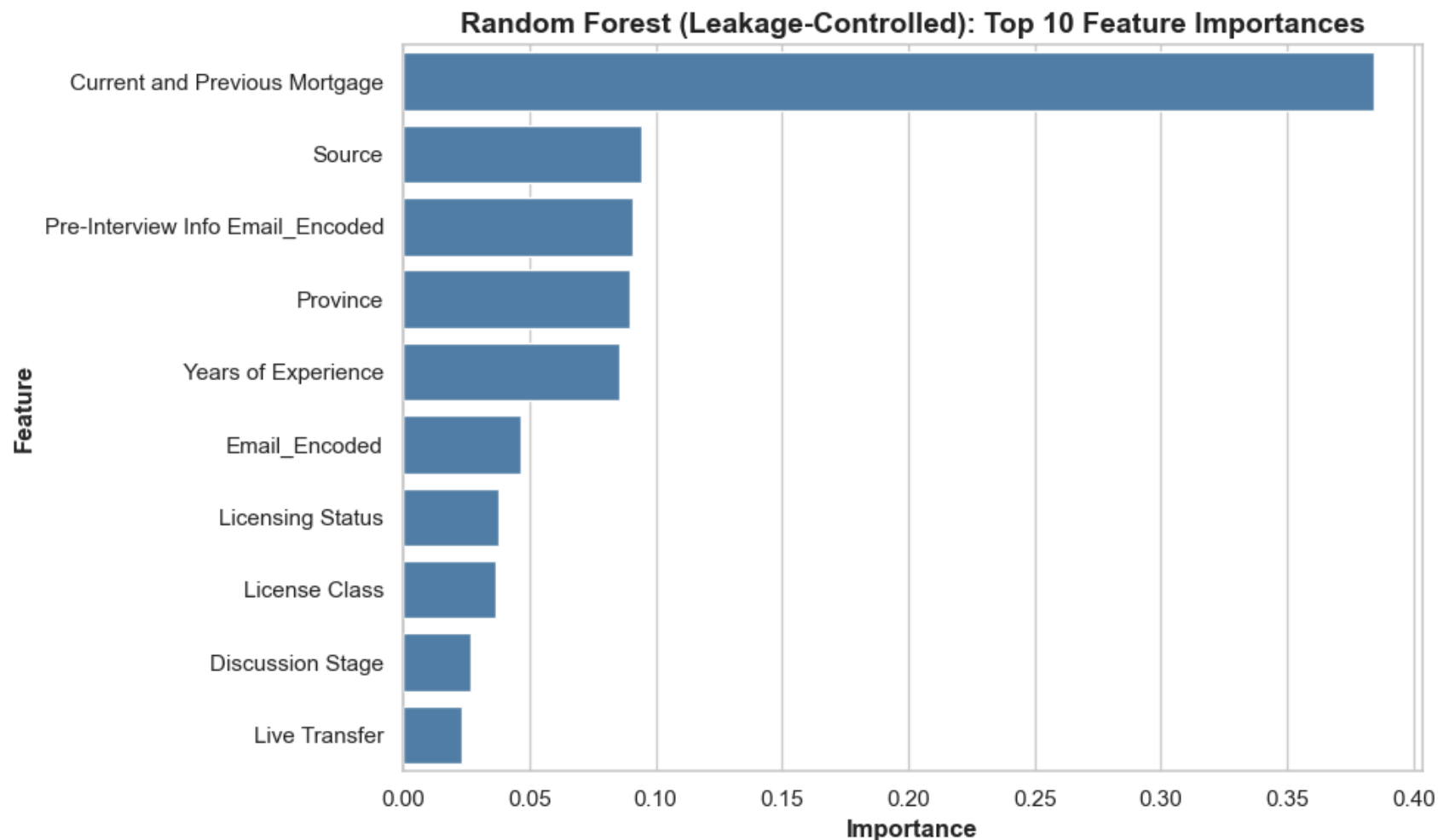
```

PR-AUC: 0.6610

	Feature	Importance
5	Current and Previous Mortgage	0.384340
1	Source	0.094554
16	Pre-Interview Info Email_Encoded	0.090833
7	Province	0.089680
4	Years of Experience	0.085725
10	Email_Encoded	0.046678
6	Licensing Status	0.037618
3	License Class	0.036353
9	Discussion Stage	0.026873
8	Live Transfer	0.023220

In [85]: *# Plotting Top 10 Importance*

```
plt.figure(figsize=(10,6))
sns.barplot(
    data=importance_clean.head(10),
    x='Importance',
    y='Feature',
    color='steelblue'
)
plt.title('Random Forest (Leakage-Controlled): Top 10 Feature Importances',
          fontsize=14, fontweight='bold')
plt.xlabel('Importance', fontsize=12, fontweight='bold')
plt.ylabel('Feature', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('feature_importance_leakage_controlled.png', dpi=150)
plt.show()
```



In [86]: *# 5-fold Cross-Validation*

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

rf_cv_original = RandomForestClassifier(random_state=RANDOM_STATE, class_weight='balanced', n_estimators=300)
cv_scores_original = cross_val_score(rf_cv_original, X_original, y, cv=skf, scoring='f1')
print(f"Original spec - RF 5-fold CV F1: {cv_scores_original.mean():.3f} +/- {cv_scores_original.std():.3f}")

rf_cv_clean = RandomForestClassifier(random_state=RANDOM_STATE, class_weight='balanced', n_estimators=300)
cv_scores_clean = cross_val_score(rf_cv_clean, X_clean, y, cv=skf, scoring='f1')
print(f"Leakage-controlled - RF 5-fold CV F1: {cv_scores_clean.mean():.3f} +/- {cv_scores_clean.std():.3f}")
```

Original spec - RF 5-fold CV F1: 0.969 +/- 0.007
 Leakage-controlled - RF 5-fold CV F1: 0.584 +/- 0.043

In [87]: # Comparison table

```
comparison_df = pd.DataFrame([
    {'Model': 'Logistic Regression', 'Original Accuracy': round(lr_original_acc, 4),
     'Original F1 (minority)': round(lr_original_f1, 4),
     'Leakage-Controlled Accuracy': round(lr_clean_acc, 4),
     'Leakage-Controlled F1 (minority)': round(lr_clean_f1, 4),
     'F1 Drop': round(lr_original_f1 - lr_clean_f1, 4)},
    {'Model': 'Decision Tree', 'Original Accuracy': round(dt_original_acc, 4),
     'Original F1 (minority)': round(dt_original_f1, 4),
     'Leakage-Controlled Accuracy': round(dt_clean_acc, 4),
     'Leakage-Controlled F1 (minority)': round(dt_clean_f1, 4),
     'F1 Drop': round(dt_original_f1 - dt_clean_f1, 4)},
    {'Model': 'Random Forest', 'Original Accuracy': round(rf_original_acc, 4),
     'Original F1 (minority)': round(rf_original_f1, 4),
     'Leakage-Controlled Accuracy': round(rf_clean_acc, 4),
     'Leakage-Controlled F1 (minority)': round(rf_clean_f1, 4),
     'F1 Drop': round(rf_original_f1 - rf_clean_f1, 4)},
])
print(comparison_df.to_string(index=False))
comparison_df.to_csv('leakage_comparison_results.csv', index=False)
importance_clean.to_csv('leakage_controlled_feature_importance.csv', index=False)
```

	Model	Original Accuracy	Original F1 (minority)	Leakage-Controlled Accuracy	Leakage-Controlled F1 (m inority)	F1 Drop
0.2480	Logistic Regression	0.9854	0.8623	0.7767	0.6143	
0.3984	Decision Tree	0.9943	0.9414	0.8787	0.5429	
0.5840	Random Forest	0.9970	0.9675	0.9595	0.3835	

The predictive analysis initially evaluated Logistic Regression, Decision Tree, and Random Forest using the original 24-feature dataset, with an 80:20 stratified train-test split. The original models produced very strong performance, with minority-class F1-scores of 0.86, 0.94 and 0.97, respectively, and Random Forest achieving the highest overall performance (99.70% accuracy and 0.9675 F1-score). However, feature-importance analysis showed that Candidate Status, Candidate Stage and Equifax Credit Score

were highly influential, raising concerns that the models were using information closely related to the outcome rather than genuinely predictive candidate characteristics.

To address this issue, a leakage-controlled analysis was conducted by removing Candidate Status, Candidate Stage, Equifax Credit Score, as well as Ethnicity, Religion and the redundant License Year variable, reducing the feature set from 24 to 18 predictors. This resulted in a substantial reduction in performance: Logistic Regression achieved 77.67% accuracy and 0.25 F1-score, Decision Tree achieved 87.87% accuracy and 0.40 F1-score, while Random Forest performed best with 95.95% accuracy and 0.58 F1-score. The Random Forest 5-fold cross-validation F1-score also decreased from 0.969 ± 0.007 in the original specification to 0.584 ± 0.043 after leakage control.

The comparison demonstrates that the exceptionally high performance of the original models was partly driven by features that introduced data leakage, rather than reflecting purely prospective predictive capability. Although the leakage-controlled models performed less strongly, their results provide a more realistic assessment of the information available for prediction and therefore offer a more defensible basis for the dissertation's conclusions. Among the three models, Random Forest remained the strongest performer after leakage control, suggesting that the remaining recruitment, communication, licensing and experience variables retain meaningful predictive information.

In []: